Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Избранные главы информатики»

ОТЧЕТ
к лабораторной работе №2
на тему:
«Работа с Docker»

БГУИР КП 6-05 0612 02 060 ПЗ

Выполнила студентка группы 453502
КРУЧЁНОК Александра Сергеевна

Проверил канд. т. н., доцент
ЖВАКИНА Анна Васильевна

_____
(дата, подпись преподавателя)

Минск 2026

Цель: Познакомиться с возможностями и получить практические навыки работы с Docker.

Вариант 13

(индивидульное задание 4)

Стек технологий: **ASP.NET MVC, IIS, Microsoft Azure, SQL Server**

Ссылки:

1. https://docs.docker.com- офф документация по докеру
2. https://hub.docker.com/ - ссылка на докерхаб
3. https://doka.guide/tools/dockerfile/ - инфа по докер файлу
4. https://k21academy.com/docker-kubernetes/docker-storage/ - инфа по хранилищу докер на англ

1. https://k21academy.com/docker-kubernetes/docker-tutorial - 5-часовой курс по докер на англ
2. https://k21academy.com/docker-kubernetes/docker-networking-different-types-of-networking-overview-for-beginners/ - инфа по сети докер на англ
3. https://ru.bitdegree.org/rukovodstvo/docker-dlja-novichkov/ - собес по докеру
4. https://habr.com/ru/companies/slurm/articles/528206/ - собес по докеру 2.0
5. https://itisgood.ru/2019/09/10/docker-setevoe-vzaimodejstvie-101/ - не пускает
6. https://itisgood.ru/2019/10/29/objasnenie-koncepcii-setej-v-docker/ - не пускает
7. https://www.tune-it.ru/web/adpashnin/blog/-/blogs/docker-network - сетевое взаимодействие докер

Задание:

1. Подготовьте рабочее окружение в соответствии с типом вашей операционной системы

- Установите Docker

Win+R, cmd – терминал → docker -v (docker --version)

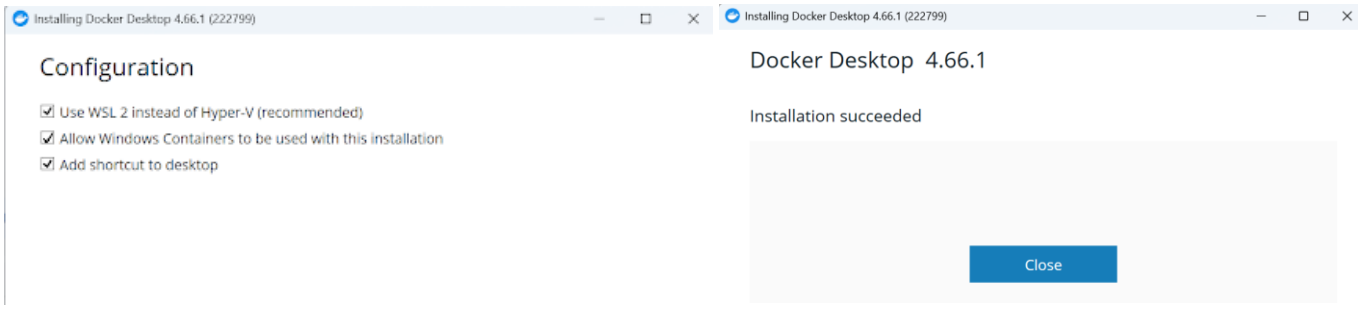
```
C:\WINDOWS\system32\cmd.exe
Microsoft Windows [Version 10.0.26100.8037]
(c) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

C:\Users\Sanechka>docker --version
"docker" не является внутренней или внешней
командой, исполняемой программой или пакетным файлом.

C:\Users\Sanechka>
```

Установка пакета Линукс на Windows

```
C:\Users\Sanechka>wsl --install
Скачивание: Ubuntu
[====                    8,4%                    ]
```

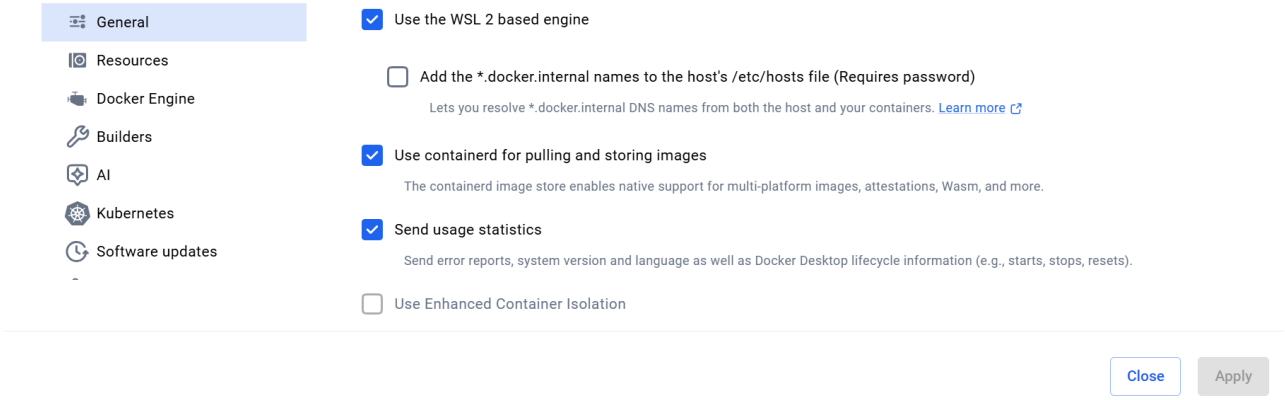
Скачивание инсталлятора Докер Десктоп для ADM64

Настройка конфигурации -> установка

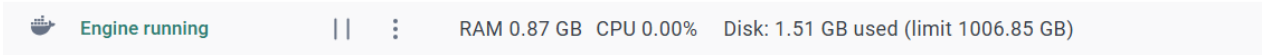


- Выполните базовую настройку

Проверка общих настроек (указывали при скачивании)



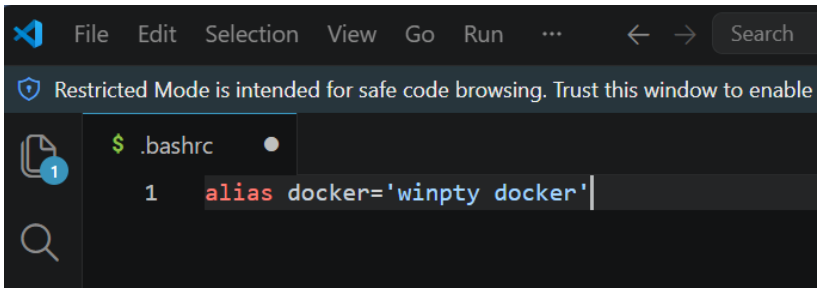
Докер успешно запущен



Настройка для работы через Git Bash
(для любых команд Docker, которые используют флаги -it (интерактивный режим и терминал), Git Bash может зависнуть или не показывать текст -> необходимо добавлять префикс winpty)

```
Sanechka@DESKTOP-81JL965 MINGW64 /
$ code ~/.bashrc

Sanechka@DESKTOP-81JL965 MINGW64 /
```

Настройка алиаса (чтобы не писать каждый раз)



Обновление (чтобы изменения вступили в силу), версия докера (установлен), тестирование работы (Docker поищет образ hello-world локально на твоем компьютере; не найдя его, он автоматически скачает его из публичного репозитория (по умолчанию — из Docker Hub, даже без аккаунта); после скачивания он запустит из этого образа контейнер; контейнер выведет на экран приветственное сообщение, которое начинается со слов "Hello from Docker!", и автоматически завершит свою работу)

```
Sanechka@DESKTOP-81JL965 MINGW64 /
$ source ~/.bashrc

Sanechka@DESKTOP-81JL965 MINGW64 /
$ docker --version
Docker version 29.3.1, build c2be9cc

Sanechka@DESKTOP-81JL965 MINGW64 /
$ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:452a468a4bf985040037cb6d5392410206e47db9bf5b7278d281f94d1c2d0931
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/


Sanechka@DESKTOP-81JL965 MINGW64 /
```

Проверка установленной версии утилиты **Docker Compose**.
обычная команда docker run (как с hello-world) запускает только *один* контейнер,
docker compose — это оркестратор для локальной разработки, кот. позволяет с помощью одного конфигурационного файла запустить сразу целую связку связанных контейнеров (например, сайт + базу данных + сервер кэширования) одной командой.

```
Sanechka@DESKTOP-81JL965 MINGW64 /
$ docker compose version
Docker Compose version v5.1.0

Sanechka@DESKTOP-81JL965 MINGW64 /
$ docker info
Client:
 Version:    29.3.1
 Context:    desktop-linux
 Debug Mode: false
 Plugins:
  agent: Docker AI Agent Runner (Docker Inc.)
    Version:  v1.32.4
    Path:     C:\Program Files\Docker\cli-plugins\docker-agent.exe
  ai: Docker AI Agent - Ask Gordon (Docker Inc.)
    Version:  v1.20.1
    Path:     C:\Program Files\Docker\cli-plugins\docker-ai.exe
  buildx: Docker Buildx (Docker Inc.)
```

2. Изучите простейшие консольные команды и возможности Docker Desktop (см. лекцию), создать собственный контейнер docker/getting-started, открыть в браузере и изучить tutorial

Создание контейнера:
run - запускаем контейнер. Если нет проекта, то докачивает и запускает (используется команда для скачивания pull контейнера) - «найди образ, создай из него контейнер и запусти»

-d - запускаем контейнер на заднем фоне (фоновый режим) - если бы мы не указали -d, терминал бы "завис", отображая логи веб-сервера, и мы не смогли бы вводить новые команды.

-p связываем порты 80:80 (сопоставить порт 80 хоста (нашей локальной машины) с портом 80 в контейнере (порт внутри приложения 80))
проброс портов. Это как пробить туннель между твоим Windows и изолированным контейнером Linux.
Первое число 80 — это порт на твоем компьютере (Windows/хост).
Второе число 80 — это порт внутри контейнера, на котором работает веб-сервер туториала.
Вся конструкция означает: «весь трафик, который приходит на порт 80 моего компьютера, перенаправляй на порт 80 внутрь этого контейнера».

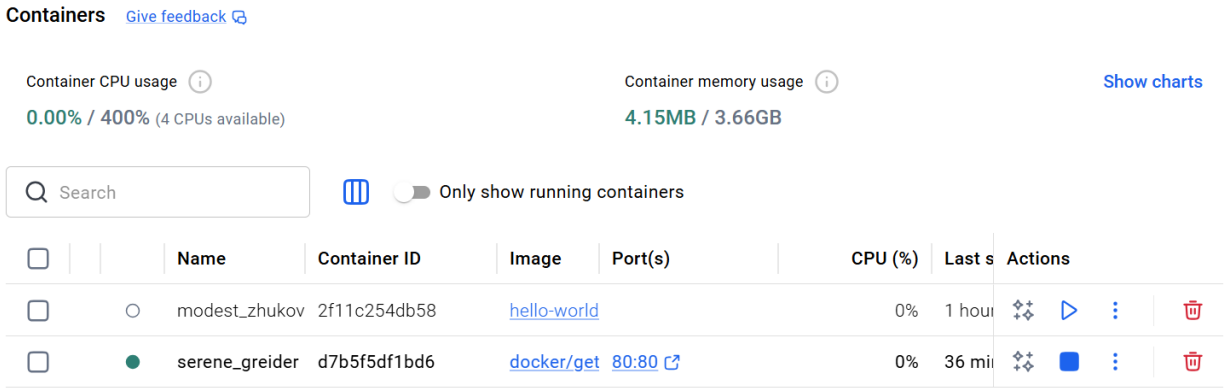
Можно -dp (сокращаем)

docker/getting-started — имя образа, который мы хотим запустить.

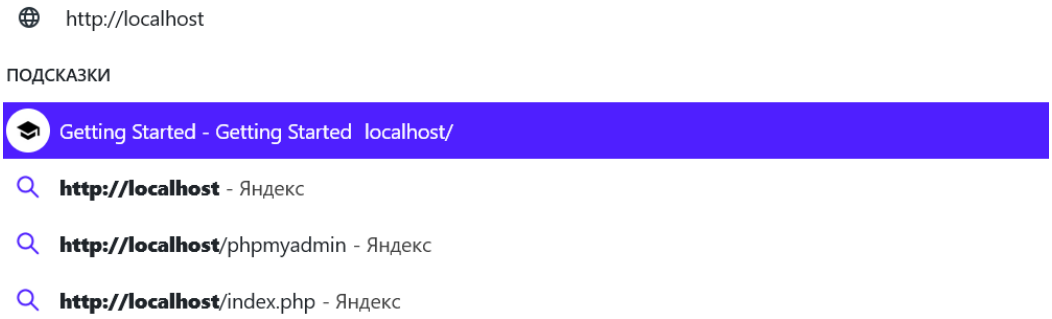
```
Sanechka@DESKTOP-81JL965 MINGW64 /
$ docker run -d -p 80:80 docker/getting-started
Unable to find image 'docker/getting-started:latest' locally
latest: Pulling from docker/getting-started
4f7e34c2de10: Pull complete
1e35f6679fab: Pull complete
c158987b0551: Pull complete
b6334b6ace34: Pull complete
9b6f639ec6ea: Pull complete
f1d1c9928c82: Pull complete
cb9626c74200: Pull complete
ee68d3549ec8: Pull complete
33e0cbbb4673: Pull complete
Digest: sha256:d79336f4812b6547a53e735480dde67f8f8f7071b414fbd9297609ffb989abc1
Status: Downloaded newer image for docker/getting-started:latest
d7b5f5df1bd66f7fe09e68edf5dee19fcc9078d0e259d76c6d112ecc61dd6225

Sanechka@DESKTOP-81JL965 MINGW64 /
```

Если возникает ошибка from daemon, то **нужно остановить службу веб-публикаций W3SVC** в диспетчере задач
ИЛИ
docker run -d -P --name docker-tutorial docker101tutorial
docker ps – чтобы узнать разрешенный порт.

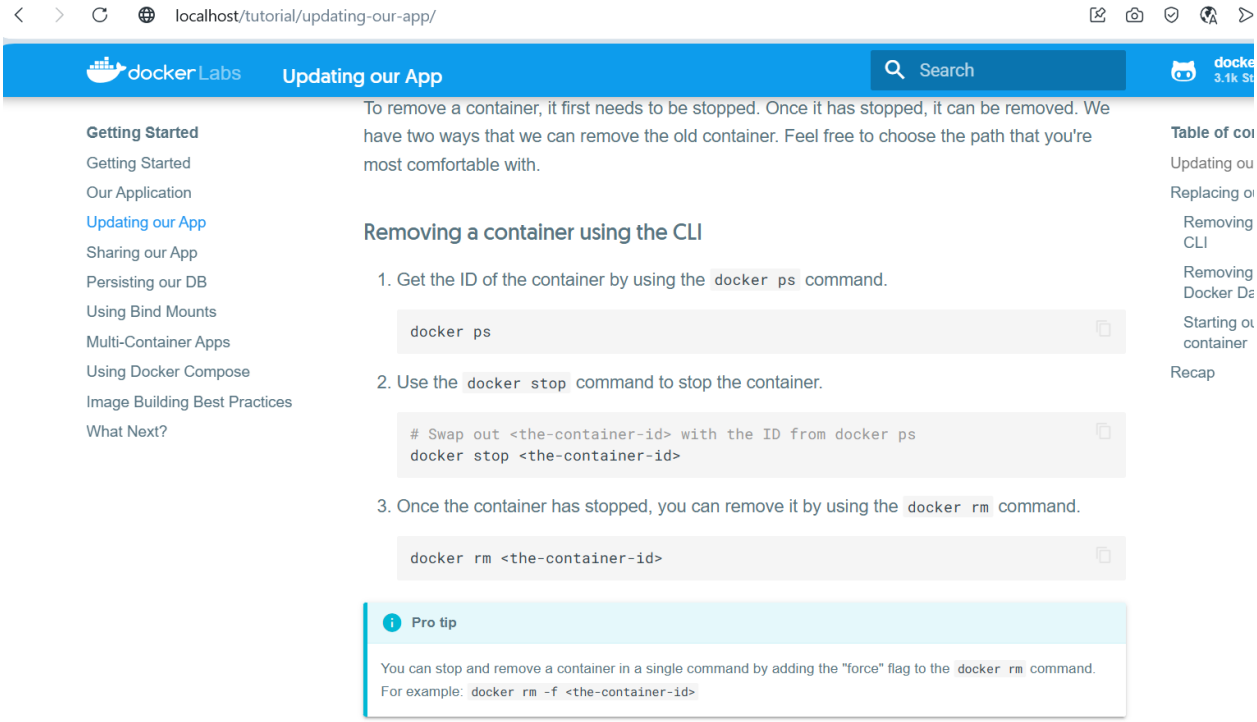
Вид в Docker Desktop:
тут можно открыть через браузер (**три точки** возле мусорки, **Open with Browser**)



Открыть в браузере (ввести http://localhost ИЛИ http://localhost:80 ИЛИ localhost ИЛИ localhost:80)
ВАЖНО: не https !



Изучение туториала:



ОБЗОР КОНТЕЙНЕРА В ПРИЛОЖЕНИИ

Клик по имени контейнера: Откроется детальное меню.

- Вкладка **Logs:** Здесь в реальном времени отображается всё, что "говорит" контейнер (например, запросы к веб-серверу, когда ты обновляешь страницу в браузере).
- Вкладка **Inspect:** Тут лежат все низкоуровневые настройки контейнера (IP-адреса, привязанные папки и т.д.).
- Вкладка **Terminal (или Exec):** Встроенная консоль *внутри* самого контейнера!
- Вкладка **Files:** Позволяет просматривать файловую систему внутри контейнера прямо как в проводнике Windows.

ОБЗОР КОНТЕЙНЕРА В ТЕРМИНАЛЕ

Просмотр запущенных контейнеров – **docker ps**

```
Sanechka@DESKTOP-81JL965 MINGW64 /
$ docker ps
CONTAINER ID   IMAGE                     COMMAND                  CREATED       STATUS          PORTS                                     NAMES
d7b5f5df1bd6   docker/getting-started   "/docker-entrypoint.…"   2 hours ago   Up 34 minutes   0.0.0.0:80->80/tcp, [::]:80->80/tcp   serene_greider

Sanechka@DESKTOP-81JL965 MINGW64 /
```

Просмотр всех контейнеров (включая остановленные остановленных) – **docker ps -a**

```
Sanechka@DESKTOP-81JL965 MINGW64 /
$ docker ps -a
CONTAINER ID   IMAGE                     COMMAND                  CREATED       STATUS                     PORTS                                     NAMES
d7b5f5df1bd6   docker/getting-started   "/docker-entrypoint.…"   2 hours ago   Up 36 minutes              0.0.0.0:80->80/tcp, [::]:80->80/tcp   serene_greider
2f11c254db58   hello-world              "/hello"                 2 hours ago   Exited (0) 2 hours ago                                           modest_zhukovsky

Sanechka@DESKTOP-81JL965 MINGW64 /
```

Просмотр всех скачанных образов(шаблонов) – **docker images**

```
Sanechka@DESKTOP-81JL965 MINGW64 /
$ docker images
                                                                                                    i Info →   U  In Use
IMAGE                          ID             DISK USAGE   CONTENT SIZE   EXTRA
docker/getting-started:latest  d79336f4812b       73.9MB         21.4MB    U
hello-world:latest             452a468a4bf9       25.9kB         9.49kB    U

Sanechka@DESKTOP-81JL965 MINGW64 /
```

Просмотр инфы о размерах контейнеров (активного и всех) – **docker ps -s**

```
Sanechka@DESKTOP-81JL965 MINGW64 /
$ docker ps -s
CONTAINER ID   IMAGE                     COMMAND                  CREATED       STATUS          PORTS                                     NAMES            SIZE
d7b5f5df1bd6   docker/getting-started   "/docker-entrypoint.…"   2 hours ago   Up 42 minutes   0.0.0.0:80->80/tcp, [::]:80->80/tcp   serene_greider   81.9kB (virtual 52.6MB
)

Sanechka@DESKTOP-81JL965 MINGW64 /
$ docker ps -as
CONTAINER ID   IMAGE                     COMMAND                  CREATED       STATUS                     PORTS                                     NAMES            SIZE
d7b5f5df1bd6   docker/getting-started   "/docker-entrypoint.…"   2 hours ago   Up 42 minutes              0.0.0.0:80->80/tcp, [::]:80->80/tcp   serene_greider   81.9kB (vir
tual 52.6MB)
2f11c254db58   hello-world              "/hello"                 3 hours ago   Exited (0) 3 hours ago                                           modest_zhukovsky   4.1kB (virt
ual 49.2kB)

Sanechka@DESKTOP-81JL965 MINGW64 /
```

В ОБРАЩЕНИИ К КОНТЕЙНЕРУ/ОБРАЗУ МОЖНО ИСПОЛЬЗОВАТЬ **ИМЯ** ИЛИ **ID**

Вывод всех логов контейнера - **docker logs** имя/айди

```
Sanechka@DESKTOP-81JL965 MINGW64 /
$ docker logs d7b5f5df1bd6
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/03/29 21:47:49 [notice] 1#1: using the "epoll" event method
2026/03/29 21:47:49 [notice] 1#1: nginx/1.23.3
2026/03/29 21:47:49 [notice] 1#1: built by gcc 12.2.1 20220924 (Alpine 12.2.1_git20220924-r4)
2026/03/29 21:47:49 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
2026/03/29 21:47:49 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/03/29 21:47:49 [notice] 1#1: start worker processes
2026/03/29 21:47:49 [notice] 1#1: start worker process 30
2026/03/29 21:47:49 [notice] 1#1: start worker process 31
2026/03/29 21:47:49 [notice] 1#1: start worker process 32
2026/03/29 21:47:49 [notice] 1#1: start worker process 33
172.17.0.1 - - [29/Mar/2026:22:03:39 +0000] "\x16\x03\x01\x06\xD2\x01\x00\x06\xCE\x03\x03\x89\x88\x81\x8F\x8Dt\xA2\x83\x84\xAAp\x92
00 157 "-" "-" "-"
172.17.0.1 - - [29/Mar/2026:22:03:39 +0000] "\x16\x03\x01\x06\xB2\x01\x00\x06\xAE\x03\x03\xDE\xDE\xA6\x96\xAF\xC5\x8F\x04Uil-\x89\x
7',\x84\x22! \xCA\x5C\xA8\x94\xE1\xEB:<<f1\xC0_\xDB}\xDA\xED\x978\xCB\xBF\xE3K\xC7\x92\xA5%\xEE\x07\xB0((\x00 zz\x13\x01\x13\x02\x1
\xC0\x13\xC0\x14\x00\x9C\x00\x9D\x00/\x005\x01\x00\x06E\xEA\xEA\x00\x00\x00+\x00\x07\x06ZZ\x03\x04\x03\x03\x00\x05\x00\x05\x01\x00\
12\x00\x00\x00#\x00\x00\x00\x1B\x00\x03\x02\x00\x02D\xCD\x00\x05\x00\x03\x02h2\x00\x10\x00\x0E\x00\x0C\x02h2\x08http/1.1\x00\x17\x0
172.17.0.1 - - [29/Mar/2026:22:03:39 +0000] "\x16\x03\x01\x06\xD2\x01\x00\x06\xCE\x03\x03\xF23r\x89\xDB\xF0\x1DS0\xF1\x81\xB5\x14,\
 B\xDFO" 400 157 "-" "-" "-"
172.17.0.1 - - [29/Mar/2026:22:03:39 +0000] "\x16\x03\x01\x06\xF2\x01\x00\x06\xEE\x03\x03\x19\x83\x81\xB8\xDF2Z\x11WZ\x81\x17\xE1\x
\x9A\xD4\xA6" 400 157 "-" "-" "-"
172.17.0.1 - - [29/Mar/2026:22:03:48 +0000] "\x16\x03\x01\x06\xF2\x01\x00\x06\xEE\x03\x03, g\xE5#M\xB7\xBEi\x9D\xAA\xEB\x8B\xC6\x9C
9\xE3\xCA \xAB\xFE\xA6" 400 157 "-" "-" "-"
172.17.0.1 - - [29/Mar/2026:22:03:48 +0000] "\x16\x03\x01\x06\xB2\x01\x00\x06\xAE\x03\x03\xEA@]F`\xA6\xCBN\x10J\x01\xFC\xCF\xD4\xD3
7\xCEq V&5H\xA7\x87\xD7<4\x1AH\x8B\xB7\xFB\x13''X\xFA\xB4\x9B?\xFD\xC7\x08\xD5@\xAE\xD3\xD1\x836\x00 \xDA\xDA\x13\x01\x13\x02\x13\x
0\x13\xC0\x14\x00\x9C\x00\x9D\x00/\x005\x01\x00\x06EZZ\x00\x00\x00-\x00\x02\x01\x01\x00\x1B\x00\x03\x02\x00\x02\x003\x04\xEF\x04\xE
4|\xAB\xDBLk\xE5\x98\xA3\x11\x8Fs\x02H\x22\xEC8\x92\xE6+\xE3\xB6\x04\xD8[\xBD\xD6\x0CKEHp+\xE1\x17\x7FY>\xCEq\xBF\xAA\xA1\xBA\x8A\x
"-"
172.17.0.1 - - [29/Mar/2026:22:03:48 +0000] "\x16\x03\x01\x06\xF2\x01\x00\x06\xEE\x03\x03\xFA\xFA\x88\xD8\x11\xBA\xCA\xF0;5\xDFP)\x
CW8 \x9C\x16\xB7S\x86\x8A&\xE3\x91*\x99\xEA~F\xBAH\xBB\xE4v\x95Udw\x80\x06/\x9B1\xEF\x9Ag\xEA\x00 ::\x13\x01\x13\x02\x13\x03\xC0+\x
\x14\x00\x9C\x00\x9D\x00/\x005\x01\x00\x06\x85\x1A\x1A\x00\x00\xFE" 400 157 "-" "-" "-"
172.17.0.1 - - [29/Mar/2026:22:03:48 +0000] "\x16\x03\x01\x06\xD2\x01\x00\x06\xCE\x03\x03?\xB2\x9D\xBC\xAB\xC8kg\x18\xCF\xFD\xE9\x1
3f\xC1\xFD\xB0\xFB pKn\x09e:\xC4_\x83[W\xD7\xA7|\xFA\x8F\xCC\xD5\x8A\x0E\x1E\xD1v]\xE1\x05\xA6\xCB\x90\xBD=\x9C\x00 zz\x13\x01\x13\
CC\xA8\xC0\x13\xC0\x14\x00\x9C\x00\x9D\x00/\x005\x01\x00\x06e\xEA\xEA\x00\x00\x00\x0B\x00\x02\x01\x00\x00\x17\x00\x00\x00+\x00\x07\
\x00\x00\x00\x00\x00\x1B\x00\x03\x02\x00\x02\xFF\x01\x00\x01\x00\x00\x10\x00\x0E\x00\x0C\x02h2\x08http/1.1\xFE" 400 157 "-" "-" "-"
172.17.0.1 - - [29/Mar/2026:22:07:04 +0000] "GET / HTTP/1.1" 200 8702 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/53
 Safari/537.36 OPR/128.0.0.0 (Edition Yx 08)" "-"
172.17.0.1 - - [29/Mar/2026:22:07:04 +0000] "GET /tutorial/ HTTP/1.1" 200 14807 "http://localhost/" "Mozilla/5.0 (Windows NT 10.0;
```

Просмотр нескольких последних логов - **docker logs -t --tail** <number> имя/айди , где

--timestamps / -t -показывать **время** (timestamp) перед каждой строчкой лога;

--tail - **количество строк для вывода** (по умолчанию - все).

```
Sanechka@DESKTOP-81JL965 MINGW64 /
$ docker logs -t --tail 5 d7b5f5df1bd6
2026-03-29T23:09:57.027693565Z 172.17.0.1 - - [29/Mar/2026:23:09:56 +0000] "GET /tutorial/persisting-our-data/ HTTP/1.1" 200 21282 "http://localhost/tutorial/sharing-our-ap
p/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0 Safari/537.36 OPR/128.0.0.0 (Edition Yx 08)" "-"
2026-03-29T23:09:57.180196670Z 172.17.0.1 - - [29/Mar/2026:23:09:57 +0000] "GET /tutorial/persisting-our-data/items-added.png HTTP/1.1" 200 63754 "http://localhost/tutorial
/persisting-our-data/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0 Safari/537.36 OPR/128.0.0.0 (Edition Yx 08)" "-"
2026-03-29T23:09:57.196733020Z 172.17.0.1 - - [29/Mar/2026:23:09:57 +0000] "GET /tutorial/persisting-our-data/dashboard-open-cli-ubuntu.png HTTP/1.1" 200 315302 "http://loc
alhost/tutorial/persisting-our-data/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0 Safari/537.36 OPR/128.0.0.0 (Editio
n Yx 08)" "-"
2026-03-29T23:13:04.528419797Z 172.17.0.1 - - [29/Mar/2026:23:13:04 +0000] "GET /tutorial/using-bind-mounts/ HTTP/1.1" 200 19493 "http://localhost/tutorial/persisting-our-d
ata/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0 Safari/537.36 OPR/128.0.0.0 (Edition Yx 08)" "-"
2026-03-29T23:13:04.638218047Z 172.17.0.1 - - [29/Mar/2026:23:13:04 +0000] "GET /tutorial/using-bind-mounts/updated-add-button.png HTTP/1.1" 200 21838 "http://localhost/tut
orial/using-bind-mounts/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0 Safari/537.36 OPR/128.0.0.0 (Edition Yx 08)" "-
"

Sanechka@DESKTOP-81JL965 MINGW64 /
```

Просмотр логов в реальном времени - **docker logs -f** имя/айди

(где **--follow / -f** - логи **в реальном времени**)

Теперь терминал "зависнет", показывая логи. Если открыть браузер и обновить страничку `http://localhost` пару раз - в терминале появятся новые строчки запросов!

Чтобы выйти из этого режима, нажми `Ctrl + C`.

```
Sanechka@DESKTOP-81JL965 MINGW64 /
$ docker logs -f d7b5f5df1bd6
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/03/29 21:47:49 [notice] 1#1: using the "epoll" event method
2026/03/29 21:47:49 [notice] 1#1: nginx/1.23.3
2026/03/29 21:47:49 [notice] 1#1: built by gcc 12.2.1 20220924 (Alpine 12.2.1_git20220924-r4)
2026/03/29 21:47:49 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
2026/03/29 21:47:49 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/03/29 21:47:49 [notice] 1#1: start worker processes
2026/03/29 21:47:49 [notice] 1#1: start worker process 30
2026/03/29 21:47:49 [notice] 1#1: start worker process 31
2026/03/29 21:47:49 [notice] 1#1: start worker process 32
2026/03/29 21:47:49 [notice] 1#1: start worker process 33
172.17.0.1 - - [29/Mar/2026:22:03:39 +0000] "\x16\x03\x01\x06\xD2\x01\x00\x06\xCE\x03\x03\x89\x88\x81\x8F\x8Dt\xA2\x83\x84\xAAp\x92\xD9\x91\x1AS\xB8\x13\xA0\xAF=\xFA\xF1" 4
00 157 "-" "-" "-"
172.17.0.1 - - [29/Mar/2026:22:03:39 +0000] "\x16\x03\x01\x06\xB2\x01\x00\x06\xAE\x03\x03\xDE\xDE\xA6\x96\xAF\xC5\x8F\x04Uil-\x89\xE3\xF4tQ\xBD\xE8\x09I\xDE\x8A\xA9X\x9B\xE
7',\x84\x22! \xCA\x5C\xA8\x94\xE1\xEB:<<f1\xC0_\xDB}\xDA\xED\x978\xCB\xBF\xE3K\xC7\x92\xA5%\xEE\x07\xB0((\x00 zz\x13\x01\x13\x02\x13\x03\xC0+\xC0/\xC0,\xC00\xCC\xA9\xCC\xA8
\xC0\x13\xC0\x14\x00\x9C\x00\x9D\x00/\x005\x01\x00\x06E\xEA\xEA\x00\x00\x00+\x00\x07\x06ZZ\x03\x04\x03\x03\x03\x00\x05\x00\x05\x01\x00\x00\x00\x00\x00\x0B\x00\x02\x01\x00\x00\x
12\x00\x00\x00#\x00\x00\x00\x1B\x00\x03\x02\x00\x02D\xCD\x00\x05\x00\x03\x02h2\x00\x10\x00\x0E\x00\x0C\x02h2\x08http/1.1\x00\x17\x00\x00\xFE" 400 157 "-" "-" "-"
172.17.0.1 - - [29/Mar/2026:22:03:39 +0000] "\x16\x03\x01\x06\xD2\x01\x00\x06\xCE\x03\x03\xF23r\x89\xDB\xF0\x1DS0\xF1\x81\xB5\x14,\xEE\xF9\xE6WR<u\xB6P6\xFC\xE9y\x04JW\xFDF
 B\xDFO" 400 157 "-" "-" "-"
172.17.0.1 - - [29/Mar/2026:22:03:39 +0000] "\x16\x03\x01\x06\xF2\x01\x00\x06\xEE\x03\x03\x19\x83\x81\xB8\xDF2Z\x11wZ\x81\x17\xE1\xAC\x88\xF6\xC6E\xEEs\xE1.\xB2\xD7\x9E\x09
\x9A\xD4\xA6" 400 157 "-" "-" "-"
172.17.0.1 - - [29/Mar/2026:22:03:48 +0000] "\x16\x03\x01\x06\xF2\x01\x00\x06\xEE\x03\x03, g\xE5#M\xB7\xBEi\x9D\xAA\xEB\x8B\xC6\x90\xF0I\x8A~\xF8\x9Fb;\xBAu\x8C\x91.\xDD\x0
9\xE3\xCA \xAB\xFE\xA6" 400 157 "-" "-" "-"
172.17.0.1 - - [29/Mar/2026:22:03:48 +0000] "\x16\x03\x01\x06\xB2\x01\x00\x06\xAE\x03\x03\xEA@]F`\xA6\xCBN\x10J\x01\xFC\xCF\xD4\xD3\x19(\xE2\xBA\xB24#\xE7\x06)w%\x18\x8F\x9
7\xCEq V&5H\xA7\x87\xD7<4\x1AH\x8B\xB7\xFB\x13''X\xFA\xB4\x9B?\xFD\xC7\x08\xD5@\xAE\xD3\xD1\x836\x00 \xDA\xDA\x13\x01\x13\x02\x13\x03\xC0+\xC0/\xC0,\xC00\xCC\xA9\xCC\xA8\xC
0\x13\xC0\x14\x00\x9C\x00\x9D\x00/\x005\x01\x00\x06EZZ\x00\x00\x00-\x00\x02\x01\x01\x00\x1B\x00\x03\x02\x00\x02\x003\x04\xEF\x04\xED\xCA\xCA\x00\x01\x00\x11\xEC\x04\xC09\xD
4|\xAB\xDBLk\xE5\x98\xA3\x11\x8Fs\x02H\x22\xEC8\x92\xE6+\xE3\xB6\x04\xD8[\xBD\xD6\x0CKEHp+\xE1\x17\x7FY>\xCEq\xBF\xAA\xA1\xBA\x8A\xFB\x13\xBD\xFA\xB5\x7FR" 400 157 "-" "-"
"-"
172.17.0.1 - - [29/Mar/2026:22:03:48 +0000] "\x16\x03\x01\x06\xF2\x01\x00\x06\xEE\x03\x03\xFA\xFA\x88\xD8\x11\xBA\xCA\xF0;5\xDFP)\xBB`*I\xE8\xC2\xFD\x88\x17',\x05w\xEB\xACc
CW8 \x9C\x16\xB7S\x86\x8A&\xE3\x91*\x99\xEA~F\xBAH\xBB\xE4v\x95Udw\x80\x06/\x9B1\xEF\x9Ag\xEA\x00 ::\x13\x01\x13\x02\x13\x03\xC0+\xC0/\xC0,\xC00\xCC\xA9\xCC\xA8\xC0\x13\xC0
\x14\x00\x9C\x00\x9D\x00/\x005\x01\x00\x06\x85\x1A\x1A\x00\x00\xFE" 400 157 "-" "-" "-"
172.17.0.1 - - [29/Mar/2026:22:03:48 +0000] "\x16\x03\x01\x06\xD2\x01\x00\x06\xCE\x03\x03?\xB2\x9D\xBC\xAB\xC8kg\x18\xCF\xFD\xE9\x1A\xBB\xD1W\xE7| \xDD\xFA\x1A\x90dm\xBB\xB
3f\xC1\xFD\xB0\xFB pKn\x09e:\xC4_\x83[W\xD7\xA7|\xFA\x8F\xCC\xD5\x8A\x0E\x1E\xD1v]\xE1\x05\xA6\xCB\x90\xBD=\x9C\x00 zz\x13\x01\x13\x02\x13\x03\xC0+\xC0/\xC0,\xC00\xCC\xA9\x
CC\xA8\xC0\x13\xC0\x14\x00\x9C\x00\x9D\x00/\x005\x01\x00\x06e\xEA\xEA\x00\x00\x00\x0B\x00\x02\x01\x00\x00\x17\x00\x00\x00+\x00\x07\x06jj\x03\x04\x03\x03\x00\x05\x00\x05\x01
\x00\x00\x00\x00\x00\x00\x1B\x00\x03\x02\x00\x02\xFF\x01\x00\x01\x00\x00\x10\x00\x0E\x00\x0C\x02h2\x08http/1.1\xFE" 400 157 "-" "-" "-"
172.17.0.1 - - [29/Mar/2026:22:07:04 +0000] "GET / HTTP/1.1" 200 8702 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0
 Safari/537.36 OPR/128.0.0.0 (Edition Yx 08)" "-"
172.17.0.1 - - [29/Mar/2026:22:07:04 +0000] "GET /tutorial/ HTTP/1.1" 200 14807 "http://localhost/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, li
```

Просмотр основные команды Docker:

```
Sanechka@DESKTOP-81JL965 MINGW64 /
$ docker
Usage:  docker [OPTIONS] COMMAND

A self-sufficient runtime for containers

Common Commands:
  run         Create and run a new container from an image
  exec        Execute a command in a running container
  ps          List containers
  build       Build an image from a Dockerfile
  bake        Build from a file
  pull        Download an image from a registry
  push        Upload an image to a registry
  images      List images
  login       Authenticate to a registry
  logout      Log out from a registry
  search      Search Docker Hub for images
  version     Show the Docker version information
  info        Display system-wide information

Management Commands:
  agent*      Docker AI Agent Runner
  ai*         Docker AI Agent - Ask Gordon
  builder     Manage builds
  buildx*     Docker Buildx
  compose*    Docker Compose
  container   Manage containers
  context     Manage contexts
  debug*      Get a shell into any image or container
  desktop*    Docker Desktop commands
  dhi*        CLI for managing Docker Hardened Images
  extension*  Manages Docker extensions
  image       Manage images
  init*       Creates Docker-related starter files for your project
  manifest    Manage Docker image manifests and manifest lists
  mcp*        Docker MCP Plugin
  model*      Docker Model Runner
  network     Manage networks
  offload*    Docker Offload
  plugin      Manage plugins
  sandbox*    Docker Sandbox
  sbom*       View the packaged-based Software Bill Of Materials (SBOM) for an image
  scout*      Docker Scout
  system      Manage Docker
  volume      Manage volumes
```

```
Swarm Commands:
  swarm       Manage Swarm

Commands:
  attach      Attach local standard input, output, and error streams to a running container
  commit      Create a new image from a container's changes
  cp          Copy files/folders between a container and the local filesystem
  create      Create a new container
  diff        Inspect changes to files or directories on a container's filesystem
  events      Get real time events from the server
  export      Export a container's filesystem as a tar archive
  history     Show the history of an image
  import      Import the contents from a tarball to create a filesystem image
  inspect     Return low-level information on Docker objects
  kill        Kill one or more running containers
  load        Load an image from a tar archive or STDIN
  logs        Fetch the logs of a container
  pause       Pause all processes within one or more containers
  port        List port mappings or a specific mapping for the container
  rename      Rename a container
  restart     Restart one or more containers
  rm          Remove one or more containers
  rmi         Remove one or more images
  save        Save one or more images to a tar archive (streamed to STDOUT by default)
  start       Start one or more stopped containers
  stats       Display a live stream of container(s) resource usage statistics
  stop        Stop one or more running containers
  tag         Create a tag TARGET_IMAGE that refers to SOURCE_IMAGE
  top         Display the running processes of a container
  unpause     Unpause all processes within one or more containers
  update      Update configuration of one or more containers
  wait        Block until one or more containers stop, then print their exit codes
```

```
Global Options:
      --config string      Location of client config files (default
                           "C:\\Users\\Sanechka\\.docker")
  -c, --context string     Name of the context to use to connect to the
                           daemon (overrides DOCKER_HOST env var and
                           default context set with "docker context use")
  -D, --debug              Enable debug mode
  -H, --host string        Daemon socket to connect to
  -l, --log-level string   Set the logging level ("debug", "info",
                           "warn", "error", "fatal") (default "info")
      --tls                Use TLS; implied by --tlsverify
      --tlscacert string   Trust certs signed only by this CA (default
                           "C:\\Users\\Sanechka\\.docker\\ca.pem")
      --tlscert string     Path to TLS certificate file (default
                           "C:\\Users\\Sanechka\\.docker\\cert.pem")
      --tlskey string      Path to TLS key file (default
                           "C:\\Users\\Sanechka\\.docker\\key.pem")
      --tlsverify          Use TLS and verify the remote
  -v, --version            Print version information and quit

Run 'docker COMMAND --help' for more information on a command.

For more help on how to use Docker, head to https://docs.docker.com/go/guides/

Sanechka@DESKTOP-81JL965 MINGW64 /
```

Работа с жизненным циклом туториала:

Поставить контейнер на паузу - **docker pause** имя/айди

(Обнови браузер — страница зависнет и будет грузиться бесконечно, сервер "заморожен").

```
Sanechka@DESKTOP-81JL965 MINGW64 /
$ docker pause d7b5f5df1bd6
d7b5f5df1bd6

Sanechka@DESKTOP-81JL965 MINGW64 /
```

Убрать контейнер с паузы - **docker unpause** имя/айди

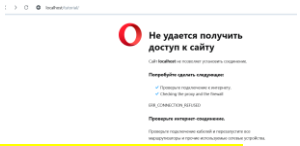
(Страница в браузере мгновенно отвиснет).

```
Sanechka@DESKTOP-81JL965 MINGW64 /
$ docker unpause d7b5f5df1bd6
d7b5f5df1bd6

Sanechka@DESKTOP-81JL965 MINGW64 /
```

Остановить контейнер - **docker stop** имя/айди

(Сервер выключен, браузер выдаст ошибку).

```
Sanechka@DESKTOP-81JL965 MINGW64 /
$ docker stop d7b5f5df1bd6
d7b5f5df1bd6

Sanechka@DESKTOP-81JL965 MINGW64 /
```



Возобновить работу контейнера - **docker start** имя/айди

(Сервер снова работает)

```
Sanechka@DESKTOP-81JL965 MINGW64 /
$ docker start d7b5f5df1bd6
d7b5f5df1bd6

Sanechka@DESKTOP-81JL965 MINGW64 /
```

Перезапустить контейнер - **docker restart** имя/айди

```
Sanechka@DESKTOP-81JL965 MINGW64 /
$ docker restart d7b5f5df1bd6
d7b5f5df1bd6

Sanechka@DESKTOP-81JL965 MINGW64 /
```

Практика по лекции

Скачать образ Ubuntu – **docker pull** ubunty

```
Sanechka@DESKTOP-81JL965 MINGW64 /
$ docker pull ubuntu
Using default tag: latest
latest: Pulling from library/ubuntu
817807f3c64e: Pull complete
e4a1e8de092c: Download complete
Digest: sha256:186072bba1b2f436cbb91ef2567abca677337cfc786c86e107d25b7072feef0c
Status: Downloaded newer image for ubuntu:latest
docker.io/library/ubuntu:latest

Sanechka@DESKTOP-81JL965 MINGW64 /
```

Запустить контейнер ИНТЕРАКТИВНО + дать ему СВОЁ имя - **docker run -it --name** MyUbuntu Ubuntu

Магия! Твой терминал Git Bash (Windows) мгновенно превратился в терминал внутри Linux Ubuntu.

```
Sanechka@DESKTOP-81JL965 MINGW64 /
$ docker run -it --name MyUbuntu ubuntu
```

Работа внутри контейнера:

Выход из контейнера: Ctrl + D (или ввести `exit` и нажми Enter) → возвращение *в Git Bash.*

Важно: при таком выходе контейнер `MyUbuntu` останавливается.

```
root@ed1ce92cf3f1:/# ls
bin   dev  home  lib64  mnt  proc  run   srv  tmp  var
boot  etc  lib   media  opt  root  sbin  sys  usr
root@ed1ce92cf3f1:/# cd media
root@ed1ce92cf3f1:/media# echo "11" > file.txt
root@ed1ce92cf3f1:/media# ls
file.txt
root@ed1ce92cf3f1:/media# cat file.txt
11
root@ed1ce92cf3f1:/media#
exit

Sanechka@DESKTOP-81JL965 MINGW64 /
```

Контейнер остановлен, но не удален (файл, который создала, всё еще там)

Запуск контейнера в фоне - **docker start** имя/айди

```
Sanechka@DESKTOP-81JL965 MINGW64 /
$ docker start MyUbuntu
MyUbuntu

Sanechka@DESKTOP-81JL965 MINGW64 /
```

Посмотрим его "паспорт" (подробные внутренние настройки в формате JSON) - **docker container inspect** MyUbuntu

```
Sanechka@DESKTOP-81JL965 MINGW64 /
$ docker container inspect MyUbuntu
[
    {
        "Id": "ed1ce92cf3f14ea86453456e3d1ecae66e6d3fd8361cbeb36d9b98cf44a335ad",
        "Created": "2026-03-30T00:10:55.185883199Z",
        "Path": "/bin/bash",
        "Args": [],
        "State": {
            "Status": "running",
            "Running": true,
            "Paused": false,
            "Restarting": false,
            "OOMKilled": false,
            "Dead": false,
            "Pid": 945,
            "ExitCode": 0,
            "Error": "",
            "StartedAt": "2026-03-30T00:12:55.667190988Z",
            "FinishedAt": "2026-03-30T00:12:21.595854828Z"
        },
        "Image": "sha256:186072bba1b2f436cbb91ef2567abca677337cfc786c86e107d25b7072feef0c",
        "ResolvConfPath": "/var/lib/docker/containers/ed1ce92cf3f14ea86453456e3d1ecae66e6d3fd8361
cbeb36d9b98cf44a335ad/resolv.conf",
        "HostnamePath": "/var/lib/docker/containers/ed1ce92cf3f14ea86453456e3d1ecae66e6d3fd8361cb
eb36d9b98cf44a335ad/hostname",
        "HostsPath": "/var/lib/docker/containers/ed1ce92cf3f14ea86453456e3d1ecae66e6d3fd8361cbeb3
6d9b98cf44a335ad/hosts",
        "LogPath": "/var/lib/docker/containers/ed1ce92cf3f14ea86453456e3d1ecae66e6d3fd8361cbeb36d
9b98cf44a335ad/ed1ce92cf3f14ea86453456e3d1ecae66e6d3fd8361cbeb36d9b98cf44a335ad-json.log",
        "Name": "/MyUbuntu",
        "RestartCount": 0,
        "Driver": "overlayfs",
        "Platform": "linux",
        "MountLabel": "",
        "ProcessLabel": "",
        "AppArmorProfile": "",
        "ExecIDs": null,
        "HostConfig": {
            "Binds": null,
            "ContainerIDFile": "",
            "LogConfig": {
                "Type": "json-file",
                "Config": {}
            },
            "NetworkMode": "bridge",
            "PortBindings": {},
            "RestartPolicy": {
```

```
            "RestartPolicy": {
                "Name": "no",
                "MaximumRetryCount": 0
            },
            "AutoRemove": false,
            "VolumeDriver": "",
            "VolumesFrom": null,
            "ConsoleSize": [
                25,
                97
            ],
            "CapAdd": null,
            "CapDrop": null,
            "CgroupnsMode": "private",
            "Dns": null,
            "DnsOptions": [],
            "DnsSearch": [],
            "ExtraHosts": null,
            "GroupAdd": null,
            "IpcMode": "private",
            "Cgroup": "",
            "Links": null,
            "OomScoreAdj": 0,
            "PidMode": "",
            "Privileged": false,
            "PublishAllPorts": false,
            "ReadonlyRootfs": false,
            "SecurityOpt": null,
            "UTSMode": "",
            "UsernsMode": "",
            "ShmSize": 67108864,
            "Runtime": "runc",
            "Isolation": "",
            "CpuShares": 0,
            "Memory": 0,
            "NanoCpus": 0,
            "CgroupParent": "",
            "BlkioWeight": 0,
            "BlkioWeightDevice": [],
            "BlkioDeviceReadBps": [],
            "BlkioDeviceWriteBps": [],
            "BlkioDeviceReadIOps": [],
            "BlkioDeviceWriteIOps": [],
            "CpuPeriod": 0,
            "CpuQuota": 0,
            "CpuRealtimePeriod": 0,
            "CpuRealtimeRuntime": 0,
            "CpusetCpus": "",
```

ОШИБКА: хочу удалить образ, но использую айди контенера

```
Sanechka@DESKTOP-81JL965 MINGW64 /
$ docker ps -a
CONTAINER ID   IMAGE                     COMMAND                  CREATED         STATUS
         PORTS                                   NAMES
ed1ce92cf3f1   ubuntu                    "/bin/bash"              5 minutes ago   Up 3 minutes
                                                 MyUbuntu
d7b5f5df1bd6   docker/getting-started    "/docker-entrypoint.…"   2 hours ago     Up 28 minutes
         0.0.0.0:80->80/tcp, [::]:80->80/tcp   serene_greider
2f11c254db58   hello-world               "/hello"                 3 hours ago     Exited (0) 3 hou
rs ago                                           modest_zhukovsky

Sanechka@DESKTOP-81JL965 MINGW64 /
$ docker image rm ed1ce92cf3f1
Error response from daemon: No such image: ed1ce92cf3f1:latest

Sanechka@DESKTOP-81JL965 MINGW64 /
```

Вывод всех скачанных образов(шаблонов) – **docker images**

```
Sanechka@DESKTOP-81JL965 MINGW64 /
$ docker images
                                                                                  i Info →   U  In Use
IMAGE                            ID             DISK USAGE   CONTENT SIZE   EXTRA
docker/getting-started:latest    d79336f4812b       73.9MB         21.4MB    U
hello-world:latest               452a468a4bf9       25.9kB         9.49kB    U
ubuntu:latest                    186072bba1b2        119MB         31.7MB    U

Sanechka@DESKTOP-81JL965 MINGW64 /
```

Копипаст нужного айди (хеллоу) и удаление - **docker image rm** 452a468a4bf9

```
Sanechka@DESKTOP-81JL965 MINGW64 /
$ docker image rm 452a468a4bf9
Error response from daemon: conflict: unable to delete 452a468a4bf9 (must be forced) - image is being used by stopped container 2f11c254
db58

Sanechka@DESKTOP-81JL965 MINGW64 /
```

Принудительное удаление образа - **docker image rm -f** 452a468a4bf9

```
Sanechka@DESKTOP-81JL965 MINGW64 /
$ docker image rm -f 452a468a4bf9
Untagged: hello-world:latest
Deleted: sha256:452a468a4bf985040037cb6d5392410206e47db9bf5b7278d281f94d1c2d0931

Sanechka@DESKTOP-81JL965 MINGW64 /
```

Удаление контейнера – **docker rm** id_контейнера

```
Sanechka@DESKTOP-81JL965 MINGW64 /
$ docker rm 2f11c254db58
2f11c254db58

Sanechka@DESKTOP-81JL965 MINGW64 /
```

Переименование существующего контейнера (тутора) – **docker rename** старое_имя новое_имя

```
Sanechka@DESKTOP-81JL965 MINGW64 /
$ docker rename serene_greider tutorial

Sanechka@DESKTOP-81JL965 MINGW64 /
```

3. Создайте docker image, который запускает скрипт с использованием функций из https://github.com/smartiqaorg/geometric_lib.

Создание папки на рабочем столе и переход в нее:

```
Sanechka@DESKTOP-81JL965 MINGW64 /
$ cd ~/Desktop

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop
```

```
Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop
$ mkdir Task3_geometric_lib

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop
$ cd Task3_geometric_lib/

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Task3_geometric_lib
```

Клонирование репы (. в конце чтобы качалось в эту же папку, а не во вложенную):

```
Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Task3_geometric_lib
$ git clone https://github.com/smartiqaorg/geometric_lib.git .
Cloning into '.'...
remote: Enumerating objects: 35, done.
remote: Total 35 (delta 0), reused 0 (delta 0), pack-reused 35 (from 1)
Receiving objects: 100% (35/35), 4.80 KiB | 289.00 KiB/s, done.
Resolving deltas: 100% (4/4), done.

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Task3_geometric_lib (main)
```

Просмотр файлов (даже скрытых), удаление .git, проверка (что удалила):

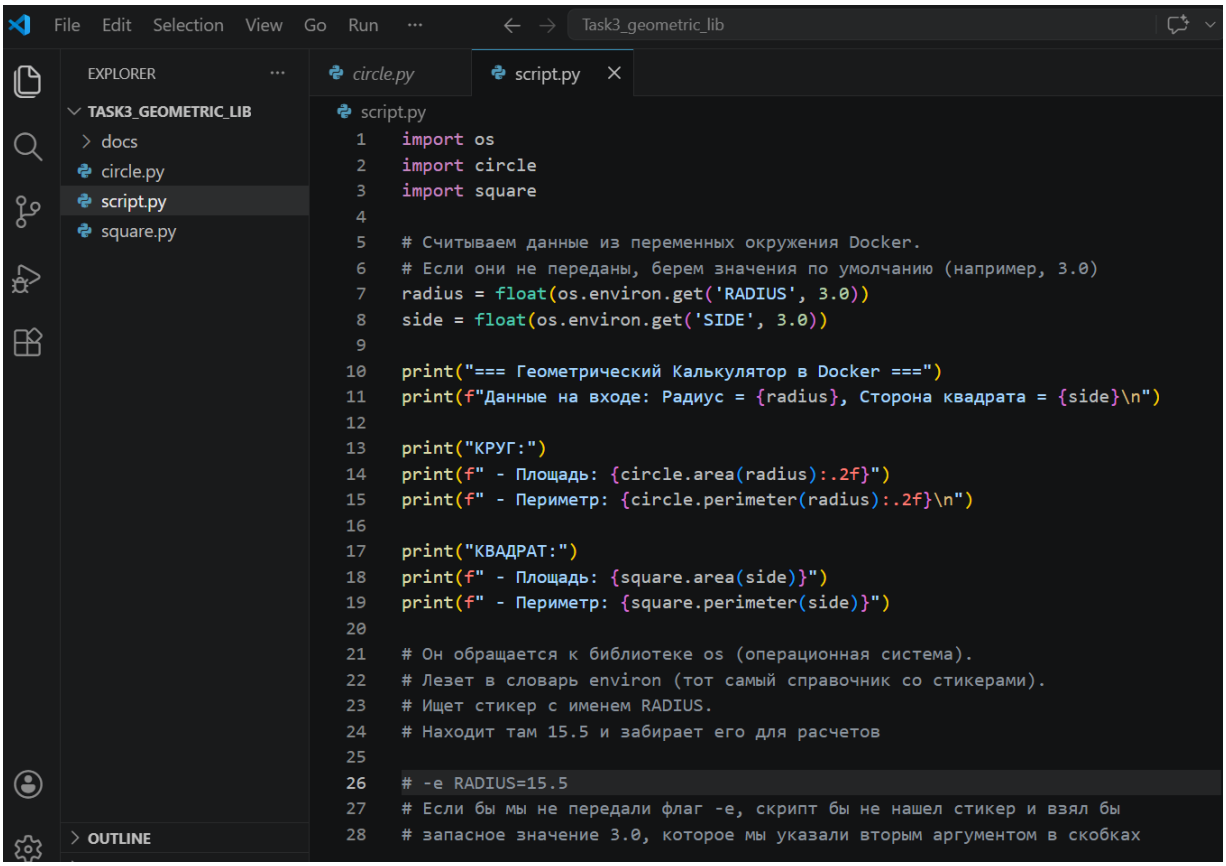
```
Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Task3_geometric_lib (main)
$ ls -A
.git/  circle.py  docs/  square.py

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Task3_geometric_lib (main)
$ rm -rf .git

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Task3_geometric_lib
$ ls -A
circle.py  docs/  square.py

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Task3_geometric_lib
```

Открытие текущей папки в VSCode:

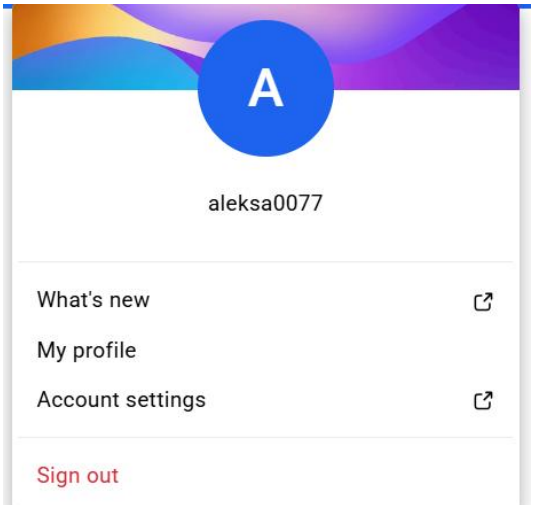
```
Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Task3_geometric_lib
$ code .

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Task3_geometric_lib
```

a. Данные необходимые для работы скрипта передайте любым удобным способом (например: конфиг файл через docker volume, **переменные окружения**, перенаправление ввода). Изучите простейшие консольные команды для работы с docker (см. лекцию). Зарегистрируйтесь на DockerHub и выберите необходимые для проекта образы

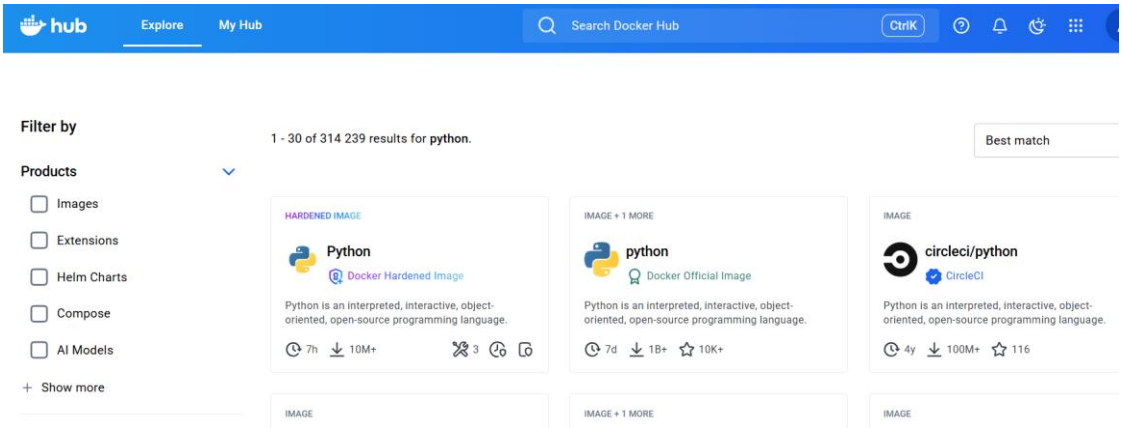
Добавляем файл-скрипт на питоне с передачей данных **через переменные окружения**: (другие способы в конце задания)



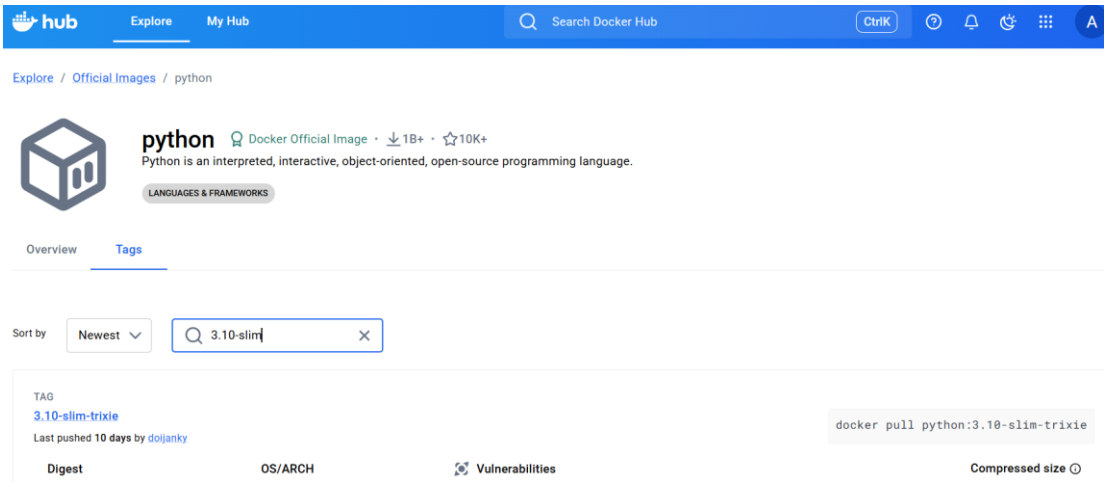
Регистрация на DockerHub:



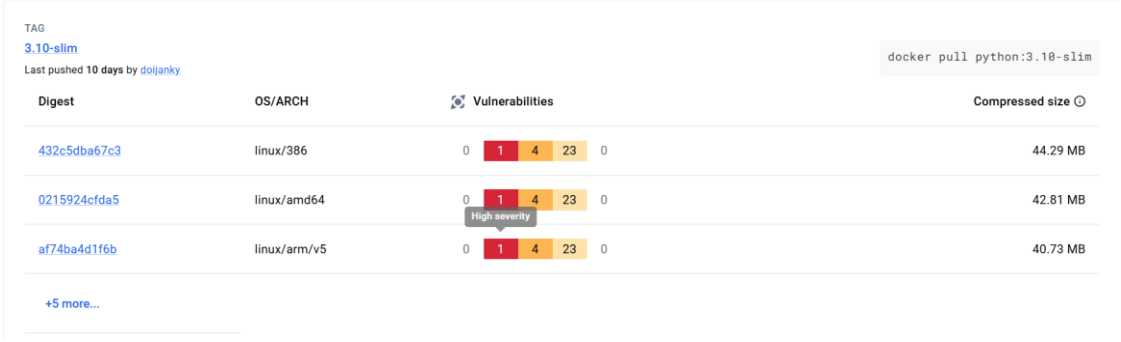
Поиск нужного питон-образа (верефецированный)



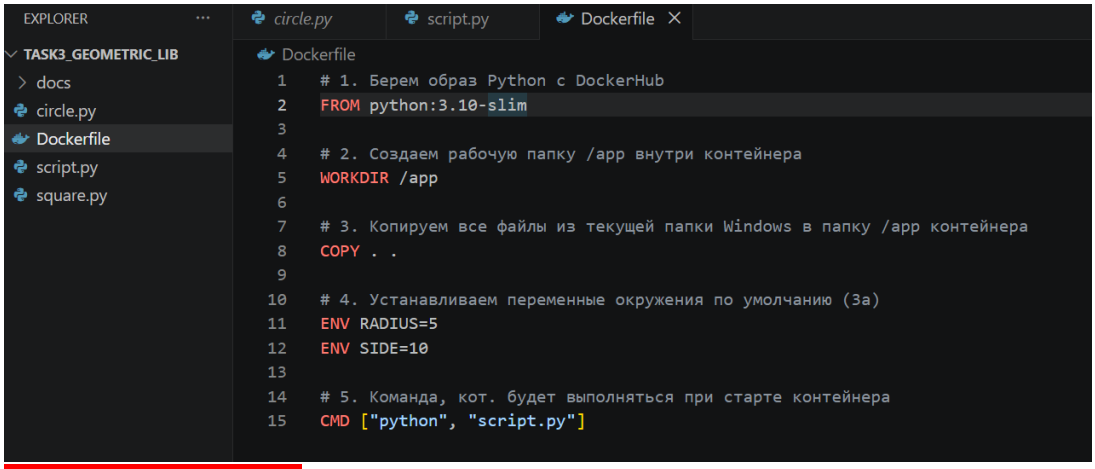
Поиск сжатой версии



Будем ее использовать:



b. Создать Dockerfile для реализации сборки собственных Docker образов



Почему `COPY`, а не `ADD`?

согласно Docker Best Practices, команда `COPY` более прозрачна для копирования локальных файлов, а магия `ADD` с распаковкой архивов нам здесь не требовалась

Почему нет `RUN`?

Нам не нужно было доустанавливать сторонние зависимости и библиотекичерез `pip`, хватило базового образа

c. Использовать его для создания контейнера. Протестировать использование контейнера

Просмотр файлов в текущей директории

```
Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Task3_geometric_lib
$ ls
Dockerfile  circle.py  docs/  script.py  square.py

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Task3_geometric_lib
```

Ошибка: неправильно указала скрипт в докерфайле, образ создался неверно →

- удаление несобравшегося контейнера через приложение
- Удаление созданного образа - **docker rmi -f** имя_образа
  (**-f** означает **принудительное удаление**; здесь необходим, т.к после неудачных попыток запуска в системе остались лежать скрытые остановленные контейнеры, кот. "держатся" за этот образ).

```
Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Task3_geometric_lib
$ docker run my-geometry-app
python: can't open file '/app/main.py': [Errno 2] No such file or directory

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Task3_geometric_lib
$ docker rmi -f my-geometry-app
Untagged: my-geometry-app:latest

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Task3_geometric_lib
```

Сборка образа - **docker build -t** имя_образа **.** (если все ок - FINISHED)

* **docker build** — команда сборки.
* **-t** my-geometry-app (tag) — мы даем нашему новому **образу имя**.
* **. (точка в конце)** — указывает Докеру искать Dockerfile в текущей папке.

```
Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Task3_geometric_lib
$ docker build -t my-geometry-app .

[+] Building 2.8s (9/9) FINISHED                                   docker:desktop-linux
 => [internal] load build definition from Dockerfile                                0.1s
 => => transferring dockerfile: 609B                                                0.0s
 => [internal] load metadata for docker.io/library/python:3.10-slim                 1.4s
 => [auth] library/python:pull token for registry-1.docker.io                       0.0s
 => [internal] load .dockerignore                                                   0.0s
 => => transferring context: 2B                                                     0.0s
 => [1/3] FROM docker.io/library/python:3.10-slim@sha256:4ba18b066cee17f2696cf9a2ba564d7d5  0.1s
 => => resolve docker.io/library/python:3.10-slim@sha256:4ba18b066cee17f2696cf9a2ba564d7d5  0.1s
 => [internal] load build context                                                   0.1s
 => => transferring context: 752B                                                   0.0s
 => CACHED [2/3] WORKDIR /app                                                       0.0s
 => [3/3] COPY . .                                                                  0.1s
 => exporting to image                                                              0.7s
 => => exporting layers                                                             0.2s
 => => exporting manifest sha256:627d8320db49f841c0901d879f49286a28a8fbc2ce893916bc89121e9  0.0s
 => => exporting config sha256:7c8d6d73b3a0ce32fbf3b2084c4c057ec4d8537518e42cc0e612b2ebe5c  0.0s
 => => exporting attestation manifest sha256:3d5c7ac4165a1ad663b6cfac900e8f8ac2d5353dff120  0.1s
 => => exporting manifest list sha256:9768ec3e54a06ea861bf4d2bcce1c6fc674ff1affa82f4798123  0.1s
 => => naming to docker.io/library/my-geometry-app:latest                           0.0s
 => => unpacking to docker.io/library/my-geometry-app:latest                        0.1s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/ed9qd9fxe338dyvs
gzr5p6nak

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Task3_geometric_lib
```

Тест 1: Запуск контейнера со значениями по умолчанию (те, что в Dockerfile)

docker run --name имя_контейнера **имя_образа**

Контейнер запустится, выполнит скрипт, выведет расчеты для радиуса 5 и стороны 10 на экран и сам завершит работу.

```
Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Task3_geometric_lib
$ docker run --name math-test1 my-geometry-app
=== Геометрический Калькулятор в Docker ===
Данные на входе: Радиус = 5.0, Сторона квадрата = 10.0

КРУГ:
 - Площадь: 78.54
 - Периметр: 31.42

КВАДРАТ:
 - Площадь: 100.0
 - Периметр: 40.0

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Task3_geometric_lib
```

Тест 2: Передача наших собственных данных снаружи

docker run --name имя_контейнера **-e RADIUS=15.5 -e SIDE=20 имя_образа**

-e - передать нужные цифры "на лету": скрипт подхватит новое и выдать уже другой результат

```
Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Task3_geometric_lib
$ docker run --name math-test2 -e RADIUS=15.5 -e SIDE=20 my-geometry-app
=== Геометрический Калькулятор в Docker ===
Данные на входе: Радиус = 15.5, Сторона квадрата = 20.0

КРУГ:
 - Площадь: 754.77
 - Периметр: 97.39

КВАДРАТ:
 - Площадь: 400.0
 - Периметр: 80.0

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Task3_geometric_lib
```

ПУШИМ НА DockerHub

Чтобы Docker Hub принял наш образ, нужно соблюсти одно важное правило: образ должен содержать в своем имени твой логин (Docker ID). Иначе Docker Hub просто не поймет, в чей аккаунт его сохранять.

Шаг 1: Тегируем (переименовываем) образ

Нам не нужно пересобирать образ заново. Докер позволяет просто "повесить" на уже существующий образ еще одну бирку (тег) с правильным именем. В консоль:

docker tag my-geometry-app aleksa0077/my-geometry-app:v1

для локальный my-geometry-app создаем ссылку с именем МОЙ_ЛОГИН/my-geometry-app и версией :v1.

```
Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Task3_geometric_lib
$ docker tag my-geometry-app aleksa0077/my-geometry-app:v1

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Task3_geometric_lib
```

Шаг 2: Авторизуемся в консоли
Консоль иногда требует подтверждения перед отправкой файлов в интернет.

docker login
Login Succeeded — всё супер.

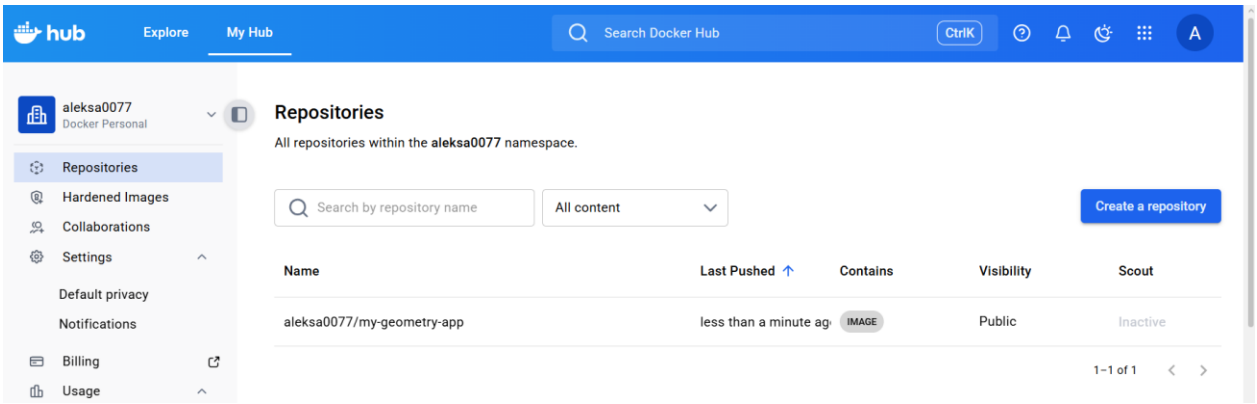
```
Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Task3_geometric_lib
$ docker login
Authenticating with existing credentials... [Username: aleksa0077]

i Info → To login with a different account, run 'docker logout' followed by 'docker login'


Login Succeeded

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Task3_geometric_lib
```

Шаг 3: Отправляем образ на Docker Hub (Push)

docker push aleksa0077/my-geometry-app:v1
В консоли появятся ползунки загрузки (Pushing). Докер начнет по кусочкам (слоям) загружать твой образ на сервер. В конце для каждого слоя → Pushed.

```
Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Task3_geometric_lib
$ docker push aleksa0077/my-geometry-app:v1
The push refers to repository [docker.io/aleksa0077/my-geometry-app]
cd9622d1a126: Pushed
bc42b1dd3474: Pushed
a5848b3a066c: Pushed
50bb60521db6: Pushed
b90ad3a1828a: Pushed
c3d226db475f: Pushed
ec781dee3f47: Pushed
v1: digest: sha256:9768ec3e54a06ea861bf4d2bcce1c6fc674ff1affa82f4798123176d7f1916ba size: 856

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Task3_geometric_lib
```

Шаг 4: Проверка
Когда загрузка завершится, зайти в браузере на hub.docker.com и перейти в раздел Repositories → репозиторий my-geometry-app появился



Теперь любой человек может написать на своем компьютере
docker pull aleksa0077/my-geometry-app:v1 и запустить мою геометрическую программу.

ВАРИАНТ ЧЕРЕЗ **конфиг-файл через Docker Volume (Тома)**

Как это работает: Ты создаешь файл с настройками в Windows. При запуске контейнера ты просишь Докер прорубить "окно" (volume) между файловой системой контейнера и твоей папкой Windows. Скрипт внутри Докера читает файл так, будто он лежит прямо у него на диске.

Шаг 1. Создаем файл конфигурации: В папке проекта (рядом с script.py) создай текстовый файл config.json и напиши там:

```
{
  "RADIUS": 8.5,
  "SIDE": 12.0
}
```

Шаг 2. Переписываем script.py: Теперь скрипт должен уметь открывать и читать этот файл.

```
import json
import circle
import square

# Открываем файл config.json и читаем из него данные
with open('config.json', 'r') as file:
    config = json.load(file)

radius = float(config.get('RADIUS', 1.0))
side = float(config.get('SIDE', 1.0))

print(f"КРУГ: Площадь = {circle.area(radius):.2f}, Периметр = {circle.perimeter(radius):.2f}")
print(f"КВАДРАТ: Площадь = {square.area(side)}, Периметр = {square.perimeter(side)}")
```

Шаг 3. Запускаем с флагом -v (Volume): Пересобери образ (docker build -t my-geometry-app .). Затем запусти команду с пробросом тома (обрати внимание на ./config.json:/app/config.json — это значит "возьми файл из текущей папки Windows и положи его в папку /app внутри контейнера"):

```
docker run -v "./config.json:/app/config.json" my-geometry-app
```

ВАРИАНТ ЧЕРЕЗ **перенаправление ввода (stdin)**

Как это работает: Скрипт просто ждет, что пользователь введет цифры с клавиатуры. Но так как мы не хотим вводить их руками в Докере, мы "выплевываем" цифры в консоли Windows и по трубе (пайпу) передаем их прямо в запущенный контейнер.

Шаг 1. Переписываем script.py: Используем стандартную функцию input().

```
import circle
import square

# Скрипт тупо ждет, что ему дадут две строчки текста (одна за другой)
print("Жду данные...")
radius = float(input())
side = float(input())

print(f"КРУГ: Площадь = {circle.area(radius):.2f}, Периметр = {circle.perimeter(radius):.2f}")
print(f"КВАДРАТ: Площадь = {square.area(side)}, Периметр = {square.perimeter(side)}")
```

Шаг 2. Запускаем с вводом (-i и echo): Пересобери образ (docker build -t my-geometry-app .).
Теперь используем флаг -i (interactive — держать канал ввода открытым) и команду echo -e, которая печатает цифры с переносом строки (\n):

```
echo -e "25.0\n40.0" | docker run -i my-geometry-app
```

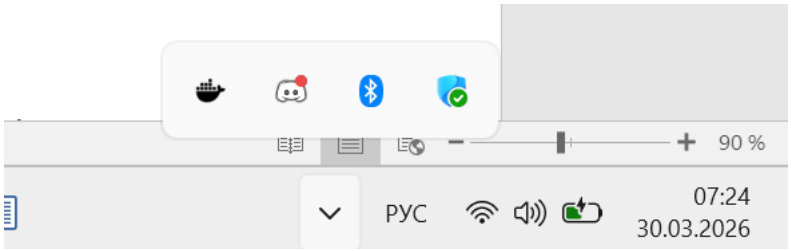
Контейнер запустится, мгновенно "проглотит" цифры 25.0 и 40.0, посчитает результат и выключится.

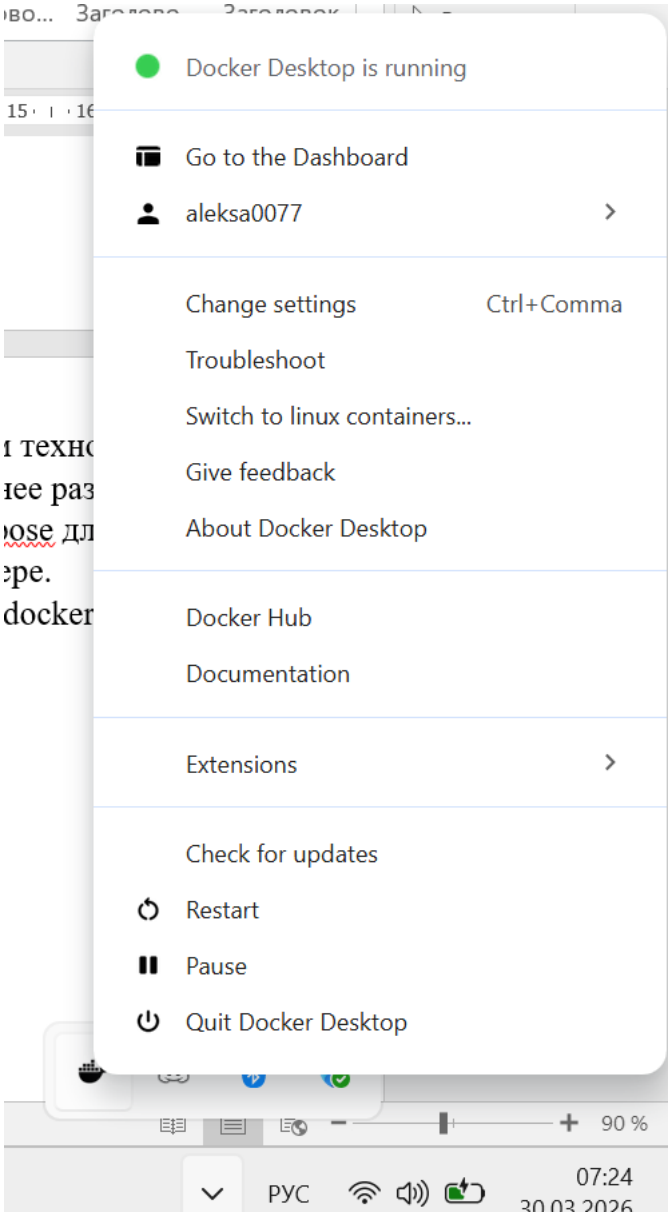
Итог:

- **Переменные окружения (-e)** — идеально для паролей, ключей и коротких настроек (как в нашей лабе).
- **Docker Volume (-v)** — идеально для огромных файлов конфигурации, картинок или баз данных.
- **Перенаправление ввода (-i)** — используется редко, в основном для скриптов автоматизации (Bash-пайплайнов).

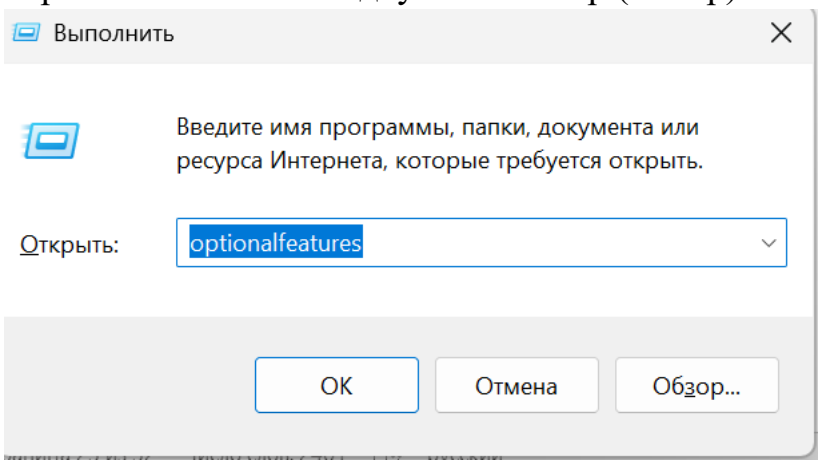
4. Скачать любой доступный проект с GitHub с произвольным стеком технологий (пример – см. индивидуальное задание) или использовать свой, ранее разработанный. Создать для него необходимый контейнер, используя Docker Compose для управления многоконтейнерными приложениями. Запустить проект в контейнере. (Примеры Images: https://hub.docker.com/_/phpmyadmin, https://hub.docker.com/_/mysql, https://hub.docker.com/_/postgres)

   Стек технологий: ASP.NET MVC, **IIS**, Microsoft Azure, SQL Server
   на винде нельзя тк майкрософт удалила все образы под скл





переключиться на виндоус контейнер (вин+р)

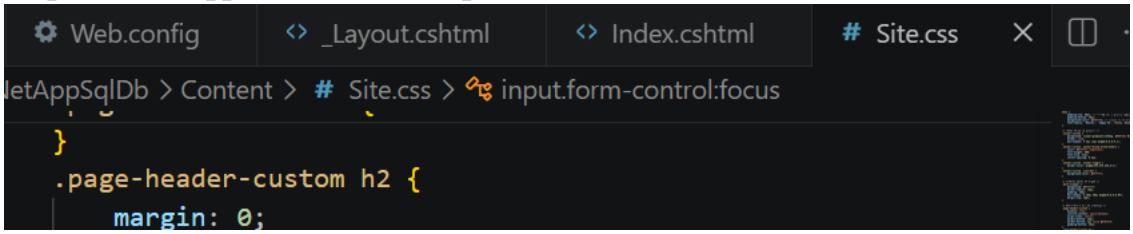


галочки напротив хупер-в и контейнеры + перезагрузка

```
Sanechka@DESKTOP-81JL965 MINGW64 /
$ cd ~/Desktop/

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop
$ mkdir Task4_DockerCompose

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop
$ cd Task4_DockerCompose/

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Task4_DockerCompose
$ git clone https://github.com/Azure-Samples/dotnet-sqldb-tutorial.git .
Cloning into '.'...
remote: Enumerating objects: 238, done.
remote: Counting objects: 100% (62/62), done.
remote: Compressing objects: 100% (16/16), done.
remote: Total 238 (delta 50), reused 46 (delta 46), pack-reused 176 (from 1)
Receiving objects: 100% (238/238), 947.83 KiB | 705.00 KiB/s, done.
Resolving deltas: 100% (118/118), done.

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Task4_DockerCompose (master)
$ ls -A
.git/  .gitignore  DotNetAppSqlDb/  DotNetAppSqlDb.sln  LICENSE  README.md

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Task4_DockerCompose (master)
$ rm -rf .git

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Task4_DockerCompose
$ ls -A
.gitignore  DotNetAppSqlDb/  DotNetAppSqlDb.sln  LICENSE  README.md

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Task4_DockerCompose
$ |
```

```
Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Task4_DockerCompose
$ code .

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Task4_DockerCompose
```

Переделала фронт и веб конфиг



		individual_task	-	-	-	2.46%	4

localhost:8080

✨ My Todo App

Мои задачи 📝

+ Создать новую

DESCRIPTION	CREATED DATE	ДЕЙСТВИЯ

© 2026 - Докер Лабораторная 🚀

```
Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Individual_Task
$ docker compose up --build -d
#1 [internal] load local bake definitions
#1 reading from stdin 547B 0.0s done
#1 DONE 0.0s

#2 [internal] load build definition from Dockerfile
#2 transferring dockerfile: 557B 0.1s done
#2 DONE 0.1s

#3 [internal] load metadata for mcr.microsoft.com/dotnet/sdk:6.0
#3 DONE 0.6s

#4 [internal] load .dockerignore
#4 transferring context: 2B done
#4 DONE 0.0s

#5 [internal] load build context
#5 DONE 0.0s

#6 [1/4] FROM mcr.microsoft.com/dotnet/sdk:6.0@sha256:c8fdd06e430de9f4ddd066b475ea350d771f341b77dd5ff4c2fafa748e3f2ef2
#6 resolve mcr.microsoft.com/dotnet/sdk:6.0@sha256:c8fdd06e430de9f4ddd066b475ea350d771f341b77dd5ff4c2fafa748e3f2ef2 0.1s done
#6 DONE 0.1s

#5 [internal] load build context
#5 transferring context: 6.45kB 0.1s done
#5 DONE 0.1s

#7 [2/4] WORKDIR /app
#7 CACHED

#8 [3/4] COPY . .
#8 DONE 0.3s

#9 [4/4] RUN dotnet restore
#9 3.223   Determining projects to restore...
#9 30.88   Restored /app/DotNetCoreSqlDb.csproj (in 26.97 sec).
#9 DONE 31.8s

#10 exporting to image
#10 exporting layers
#10 exporting layers 14.6s done
#10 exporting manifest sha256:f5098a31980201659298ed08a8d7afb5f3b55e95a1e613f8246d89aac6511b89 0.1s done
#10 exporting config sha256:40b3accce78098c98ae9d56b698e9336cf9452d06da2d38299c3c4f3e067180d 0.1s done
#10 exporting attestation manifest sha256:bb5a2b5452666259726 5af85094817bcd7ae6da01810e8c237319fbd3e0324bf
#10 exporting attestation manifest sha256:bb5a2b5452666259726 5af85094817bcd7ae6da01810e8c237319fbd3e0324bf 0.2s done
#10 exporting manifest list sha256:bd863980d8b058c08dc6e0950a8f7c56468056bef5866a0915174d065478500a
#10 exporting manifest list sha256:bd863980d8b058c08dc6e0950a8f7c56468056bef5866a0915174d065478500a 0.1s done
#10 naming to docker.io/library/individual_task-web:latest 0.1s done
#10 unpacking to docker.io/library/individual_task-web:latest
#10 unpacking to docker.io/library/individual_task-web:latest 7.5s done
#10 DONE 22.8s

#11 resolving provenance for metadata file
#11 DONE 0.1s
[+] up 3/3
 ✔ Image individual_task-web  Built
 ✔ Container sql_server_linux Running
 ✔ Container mvc_core_app     Started

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Individual_Task
$ docker compose ps
NAME              IMAGE                                   COMMAND                  SERVICE   CREATED          STATUS          PORTS
mvc_core_app      individual_task-web                     "dotnet run"             web       41 seconds ago   Up 38 seconds   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
sql_server_linux  mcr.microsoft.com/mssql/server:2022-latest  "/opt/mssql/bin/laun…"  db        8 minutes ago    Up 8 minutes    0.0.0.0:1433->1433/tcp, [::]:1433->1433/tcp

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Individual_Task
```

5. Настроить сети и тома для обеспечения связи между контейнерами и сохранения данных (исходные данные, логин, пароль и т.д.)

было

```
services:
  db:
    image: mcr.microsoft.com/mssql/server:2022-latest
    container_name: sql_server_linux
    environment:
      - ACCEPT_EULA=Y
      - MSSQL_SA_PASSWORD=SuperSecretPassword123!
    ports:
      - "1433:1433"
```

volumes:

- sql_data:/var/opt/mssql

web:

build: .

container_name: mvc_core_app

ports:

- "8080:80"

environment:

.NET Core автоматически заменит этими значениями то, что написано в appsettings.json

- ConnectionStrings__MyDbConnection=Server=db;Database=AppDb;User Id=sa;Password=SuperSecretPassword123!;TrustServerCertificate=True;

- ConnectionStrings__DefaultConnection=Server=db;Database=AppDb;User Id=sa;Password=SuperSecretPassword123!;TrustServerCertificate=True;

depends_on:

- db

volumes:

sql_data:

Изменение докер компоуз

```
services:

  db:
    image: mcr.microsoft.com/mssql/server:2022-latest
    container_name: sql_server_linux
    environment:
      - ACCEPT_EULA=Y
      # Берем пароль из файла .env
      - MSSQL_SA_PASSWORD=${DB_PASSWORD}
    ports:
      - "1433:1433"
    volumes:
      # Том для сохранения данных БД при перезапуске
      - sql_data:/var/opt/mssql
    networks:
      # Подключаем к общей сети
      - todo_network

  web:
```

```yaml
    build: .
    container_name: mvc_core_app
    ports:
      - "8080:80"
    environment:
      # Подставляем пароль из .env в строку подключения
      - ConnectionStrings__MyDbConnection=Server=db;Database=AppDb;User
Id=sa;Password=${DB_PASSWORD};TrustServerCertificate=True;
    depends_on:
      - db
    networks:
      # Подключаем к той же сети, чтобы web видел db
      - todo_network

# Явное объявление томов
volumes:
  sql_data:

# Явное объявление сети
networks:
  todo_network:
    driver: bridge
```

```
Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Individual_Task
$ docker compose ps
NAME              IMAGE                                       COMMAND                  SERVICE   CREATED         STATUS         PORTS
mvc_core_app      individual_task-web                         "dotnet run"             web       7 seconds ago   Up 5 seconds   0.0.0.0:8080->80/tcp, [::]:
8080->80/tcp
sql_server_linux  mcr.microsoft.com/mssql/server:2022-latest  "/opt/mssql/bin/laun…"   db        9 seconds ago   Up 6 seconds   0.0.0.0:1433->1433/tcp, [::
]:1433->1433/tcp

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Individual_Task
$ docker compose down -v
[+] down 3/3
 ✔ Container mvc_core_app          Removed                                                                                    2.2s
 ✔ Container sql_server_linux      Removed                                                                                    2.2s
 ✔ Volume individual_task_sql_data Removed                                                                                    0.2s

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Individual_Task
$ docker compose up -d
[+] up 4/4
 ✔ Network individual_task_todo_network Created                                                                               0.2s
 ✔ Volume individual_task_sql_data      Created                                                                               0.1s
 ✔ Container sql_server_linux           Started                                                                               2.4s
 ✔ Container mvc_core_app               Started                                                                               2.1s

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Individual_Task
$ docker compose ps
NAME              IMAGE                                       COMMAND                  SERVICE   CREATED         STATUS         PORTS
mvc_core_app      individual_task-web                         "dotnet run"             web       17 seconds ago  Up 15 seconds  0.0.0.0:8080->80/tcp, [::
]:8080->80/tcp
sql_server_linux  mcr.microsoft.com/mssql/server:2022-latest  "/opt/mssql/bin/laun…"   db        17 seconds ago  Up 15 seconds  0.0.0.0:1433->1433/tcp, [
::]:1433->1433/tcp

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Individual_Task
```

6. Разместите результат в созданный репозиторий в DockerHub

```
Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Individual_Task
$ docker login
Authenticating with existing credentials... [Username: aleksa0077]

i Info → To login with a different account, run 'docker logout' followed by 'docker login'


Login Succeeded

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Individual_Task
$ docker images

IMAGE                                          ID             DISK USAGE   CONTENT SIZE   EXTRA
aleksa0077/my-geometry-app:v1                  9768ec3e54a0        183MB         44.9MB    U
docker/getting-started:latest                  d79336f4812b       73.9MB         21.4MB    U
individual_task-web:latest                     bbd85269b4a0       1.43GB          425MB    U
mcr.microsoft.com/mssql/server:2022-latest     49b45a911dc5       2.27GB          604MB    U
my-geometry-app:latest                         9768ec3e54a0        183MB         44.9MB    U
task4-web:latest                               c87e9136fe8c       1.06GB          278MB
ubuntu:latest                                  186072bba1b2        119MB         31.7MB    U

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Individual_Task
$ docker tag individual_task-web:latest aleksa0077/todo-app:v1

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Individual_Task
$ docker push  aleksa0077/todo-app:v1
The push refers to repository [docker.io/aleksa0077/todo-app]
abf596e283b8: Pushed
78a15cfd7a7a: Pushed
b25595df572c: Pushed
89320e7119e2: Pushed
8715c03abd5b: Pushed
126dbb7122a6: Pushed
521bfe36b12b: Pushed
56efa3ddd819: Pushed
7450cfae47eb: Pushed
17701958766f: Pushed
952b1bd0bb13: Pushed
78f394ff90b4: Pushed
v1: digest: sha256:bbd85269b4a048f4f0e7f5906394af9f90878a9d7f5c16bea221f2d3455739d2 size: 856

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Individual_Task
```

7. Выполните следующие действия с целью изучить особенности сетевого взаимодействия:
   - Получить информацию о всех сетях, работающих на текущем хосте и подробности о каждом типе сети

```
Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Individual_Task
$ docker network ls
NETWORK ID     NAME                           DRIVER    SCOPE
53cbd2c87b9a   bridge                         bridge    local
21bb1ba30e72   host                           host      local
e536e72f34db   individual_task_default        bridge    local
5f8c52d45510   individual_task_todo_network   bridge    local
37b816ed1d5d   none                           null      local

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Individual_Task
$ docker network inspect bridge
[
    {
        "Name": "bridge",
        "Id": "53cbd2c87b9a289bbb9e017c956e3944b63ab3c3eed79b87aa22ae6a90d8331c",
        "Created": "2026-03-30T04:59:30.460858818Z",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv4": true,
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": null,
            "Config": [
                {
                    "Subnet": "172.17.0.0/16",
                    "Gateway": "172.17.0.1"
                }
            ]
        },
```

- Создать свою собственную сеть bridge, проверить, создана ли она, запустить Docker-контейнер в созданной сети, вывести о ней всю информацию (включая IP-адрес контейнера), отключить сеть от контейнера

```
Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Individual_Task
$ docker network create --driver bridge my_bridge_1
f8bfd622a285874de9646cc9d2aaa29115eb831143d8fd896c3d7e6eeea20340

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Individual_Task
$ docker network ls
NETWORK ID     NAME                           DRIVER    SCOPE
53cbd2c87b9a   bridge                         bridge    local
21bb1ba30e72   host                           host      local
e536e72f34db   individual_task_default        bridge    local
5f8c52d45510   individual_task_todo_network   bridge    local
f8bfd622a285   my_bridge_1                    bridge    local
37b816ed1d5d   none                           null      local

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Individual_Task
$ docker run -d --name test_container --network my_bridge_1 alpine sleep 3600
Unable to find image 'alpine:latest' locally
latest: Pulling from library/alpine
589002ba0eae: Pull complete
9e595aac14e0: Download complete
caa817ad3aea: Download complete
Digest: sha256:25109184c71bdad752c8312a8623239686a9a2071e8825f20acb8f2198c3f659
Status: Downloaded newer image for alpine:latest
6a1f8d8f2f56d9b710cca72da197fb084f3d10ca6ddec91f3f4ebc44ba45b95c

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Individual_Task
$ docker network inspect my_bridge_1
[
    {
        "Name": "my_bridge_1",
        "Id": "f8bfd622a285874de9646cc9d2aaa29115eb831143d8fd896c3d7e6eeea20340",
        "Created": "2026-03-30T07:29:18.590953333Z",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv4": true,
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": {},
            "Config": [
                {
                    "Subnet": "172.20.0.0/16",
                    "Gateway": "172.20.0.1"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Options": {
            "com.docker.network.enable_ipv4": "true",
            "com.docker.network.enable_ipv6": "false"
        },
        "Labels": {},
        "Containers": {
```

```
        "ConfigOnly": false,
        "Options": {
            "com.docker.network.enable_ipv4": "true",
            "com.docker.network.enable_ipv6": "false"
        },
        "Labels": {},
        "Containers": {
            "6a1f8d8f2f56d9b710cca72da197fb084f3d10ca6ddec91f3f4ebc44ba45b95c": {
                "Name": "test_container",
                "EndpointID": "ea74946e5c76bde0688844ed0bc212ebaf89a923f09df5a739ef95c5f11f5a8d",
                "MacAddress": "ce:ac:0f:b4:d5:46",
                "IPv4Address": "172.20.0.2/16",
                "IPv6Address": ""
            }
        },
        "Status": {
            "IPAM": {
                "Subnets": {
                    "172.20.0.0/16": {
                        "IPsInUse": 4,
                        "DynamicIPsAvailable": 65532
                    }
                }
            }
        }
    }
]

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Individual_Task
$ docker network disconnect my_bridge_1 test_container

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Individual_Task
$ docker network create --driver bridge my_bridge_2
```

- Создать еще одну сеть bridge, вывести о ней всю информацию, запустить в ней три контейнера, подключиться к любому из контейнеров и пропинговать два других из оболочки контейнера, убедиться, что между контейнерами происходит общение по IP-адресу

```
Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Individual_Task
$ docker network create --driver bridge my_bridge_2
f52ecdb849b5726fad22aa5c8bf9f8edf463014482b4e557eb6de54464b6bc29

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Individual_Task
$ docker run -d --name node1 --network my_bridge_2 alpine sleep 3600
b5fe18ab2082eb9749128e0f3686c625e01d3310c1fcef8968e1200aef71bf9a

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Individual_Task
$ docker run -d --name node2 --network my_bridge_2 alpine sleep 3600
7ffa76598386eec92a84d2b737c6bd5a9ce6a60b7cb2c86bf132914c291d9f01

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Individual_Task
$ docker run -d --name node3 --network my_bridge_2 alpine sleep 3600
091dcb1f6858d06bcc2585969f0dcd5436750f5613d3bed908ec9b1889fdd92c

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Individual_Task
$ docker network inspect my_bridge_2
[
    {
        "Name": "my_bridge_2",
        "Id": "f52ecdb849b5726fad22aa5c8bf9f8edf463014482b4e557eb6de54464b6bc29",
        "Created": "2026-03-30T07:33:40.858401744Z",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv4": true,
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": {},
            "Config": [
                {
                    "Subnet": "172.21.0.0/16",
                    "Gateway": "172.21.0.1"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Options": {
            "com.docker.network.enable_ipv4": "true",
            "com.docker.network.enable_ipv6": "false"
        },
        "Labels": {},
        "Containers": {
            "091dcb1f6858d06bcc2585969f0dcd5436750f5613d3bed908ec9b1889fdd92c": {
                "Name": "node3",
                "EndpointID": "ffd334b2b9ca9d2186aeb1a5d74fca4886a4e0fac8cfbfd253e23f1a1f17fc07",
                "MacAddress": "b2:09:6b:15:4d:2e",
                "IPv4Address": "172.21.0.4/16",
                "IPv6Address": ""
            },
            "7ffa76598386eec92a84d2b737c6bd5a9ce6a60b7cb2c86bf132914c291d9f01": {
                "Name": "node2",
                "EndpointID": "27c87b1ab59d2fe554a3f1881a6fa7559c744fa57b289dc9eb92e54df5a07844",
                "MacAddress": "6e:c4:5c:a2:cb:10",
                "IPv4Address": "172.21.0.3/16",
                "IPv6Address": ""
            },
            "b5fe18ab2082eb9749128e0f3686c625e01d3310c1fcef8968e1200aef71bf9a": {
                "Name": "node1",
                "EndpointID": "ca94755ce0166d969a158a1aecc5e9f14e8e31402a3423e8c1f38911d27c413b",
                "MacAddress": "de:fb:3c:d7:5e:de",
```

```
            "com.docker.network.enable_ipv4": "true",
            "com.docker.network.enable_ipv6": "false"
        },
        "Labels": {},
        "Containers": {
            "091dcb1f6858d06bcc2585969f0dcd5436750f5613d3bed908ec9b1889fdd92c": {
                "Name": "node3",
                "EndpointID": "ffd334b2b9ca9d2186aeb1a5d74fca4886a4e0fac8cfbfd253e23f1a1f17fc07",
                "MacAddress": "b2:09:6b:15:4d:2e",
                "IPv4Address": "172.21.0.4/16",
                "IPv6Address": ""
            },
            "7ffa76598386eec92a84d2b737c6bd5a9ce6a60b7cb2c86bf132914c291d9f01": {
                "Name": "node2",
                "EndpointID": "27c87b1ab59d2fe554a3f1881a6fa7559c744fa57b289dc9eb92e54df5a07844",
                "MacAddress": "6e:c4:5c:a2:cb:10",
                "IPv4Address": "172.21.0.3/16",
                "IPv6Address": ""
            },
            "b5fe18ab2082eb9749128e0f3686c625e01d3310c1fcef8968e1200aef71bf9a": {
                "Name": "node1",
                "EndpointID": "ca94755ce0166d969a158a1aecc5e9f14e8e31402a3423e8c1f38911d27c413b",
                "MacAddress": "de:fb:3c:d7:5e:de",
                "IPv4Address": "172.21.0.2/16",
                "IPv6Address": ""
            }
        },
        "Status": {
            "IPAM": {
                "Subnets": {
                    "172.21.0.0/16": {
                        "IPsInUse": 6,
                        "DynamicIPsAvailable": 65530
                    }
                }
            }
        }
    }
]

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Individual_Task
$ docker exec -it node1 sh
/ # ping -c 4 172.21.0.4/16
ping: bad address '172.21.0.4/16'
/ # ping -c 4 172.21.0.4
PING 172.21.0.4 (172.21.0.4): 56 data bytes
64 bytes from 172.21.0.4: seq=0 ttl=64 time=3.317 ms
64 bytes from 172.21.0.4: seq=1 ttl=64 time=0.162 ms
64 bytes from 172.21.0.4: seq=2 ttl=64 time=0.164 ms
64 bytes from 172.21.0.4: seq=3 ttl=64 time=0.181 ms

--- 172.21.0.4 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 0.162/0.956/3.317 ms
/ # ping -c 4 172.21.0.3
PING 172.21.0.3 (172.21.0.3): 56 data bytes
64 bytes from 172.21.0.3: seq=0 ttl=64 time=0.529 ms
64 bytes from 172.21.0.3: seq=1 ttl=64 time=0.145 ms
64 bytes from 172.21.0.3: seq=2 ttl=64 time=0.284 ms
64 bytes from 172.21.0.3: seq=3 ttl=64 time=0.155 ms

--- 172.21.0.3 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 0.145/0.278/0.529 ms
/ #

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Individual_Task
```

- Создать свою собственную сеть overlay, проверить, создана ли она, вывести о ней всю информацию

```
Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Individual_Task
$ docker swarm init
Swarm initialized: current node (n3rs6cafudcbnclf99v3edn4b) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-1rrgwiztjq0opi4cwlwypbgg8molt5jr2ih8bfz2b8om9x0nre-74hpma66lqwkeb5ua0odayg8d 192.168.65.3:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Individual_Task
$ docker network create -d overlay my-overlay-1
```

```
Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Individual_Task
$ docker network create -d overlay my_overlay_1
u5otskn41zhxrau10oliwwzhe

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Individual_Task
$ docker network ls
NETWORK ID     NAME                           DRIVER    SCOPE
53cbd2c87b9a   bridge                         bridge    local
3682c6791f13   docker_gwbridge                bridge    local
21bb1ba30e72   host                           host      local
e536e72f34db   individual_task_default        bridge    local
5f8c52d45510   individual_task_todo_network   bridge    local
qdg20w2cc8qq   ingress                        overlay   swarm
f8bfd622a285   my_bridge_1                    bridge    local
f52ecdb849b5   my_bridge_2                    bridge    local
u5otskn41zhx   my_overlay_1                   overlay   swarm
37b816ed1d5d   none                           null      local

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Individual_Task
$ docker network inspect my_overlay_1
[
    {
        "Name": "my_overlay_1",
        "Id": "u5otskn41zhxrau10oliwwzhe",
        "Created": "2026-03-30T07:49:30.400629074Z",
        "Scope": "swarm",
        "Driver": "overlay",
        "EnableIPv4": true,
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": null,
            "Config": [
                {
                    "Subnet": "10.0.1.0/24",
                    "Gateway": "10.0.1.1"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Options": {
            "com.docker.network.driver.overlay.vxlanid_list": "4097"
        },
        "Labels": null,
        "Containers": {},
        "Status": {
            "IPAM": {
                "Subnets": {
                    "10.0.1.0/24": {
                        "IPsInUse": 3,
                        "DynamicIPsAvailable": 253
                    }
                }
            }
        }
    }
]

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Individual_Task
```

- Создать еще одну сеть overlay, проверить, создана ли она, вывести о ней всю информацию, удалить сеть

```
Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Individual_Task
$ docker network create -d overlay my_overlay_2
yds6apv5s2owkbl403bymby7w

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Individual_Task
$ docker network ls
NETWORK ID     NAME                          DRIVER    SCOPE
53cbd2c87b9a   bridge                        bridge    local
3682c6791f13   docker_gwbridge               bridge    local
21bb1ba30e72   host                          host      local
e536e72f34db   individual_task_default       bridge    local
5f8c52d45510   individual_task_todo_network  bridge    local
qdg20w2cc8qq   ingress                       overlay   swarm
f8bfd622a285   my_bridge_1                   bridge    local
f52ecdb849b5   my_bridge_2                   bridge    local
u5otskn41zhx   my_overlay_1                  overlay   swarm
yds6apv5s2ow   my_overlay_2                  overlay   swarm
37b816ed1d5d   none                          null      local

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Individual_Task
$ docker network inspect my_overlay_2
[
    {
        "Name": "my_overlay_2",
        "Id": "yds6apv5s2owkbl403bymby7w",
        "Created": "2026-03-30T07:50:20.603506446Z",
        "Scope": "swarm",
        "Driver": "overlay",
        "EnableIPv4": true,
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": null,
            "Config": [
                {
                    "Subnet": "10.0.2.0/24",
                    "Gateway": "10.0.2.1"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Options": {
            "com.docker.network.driver.overlay.vxlanid_list": "4098"
        },
        "Labels": null,
        "Containers": {},
        "Status": {
            "IPAM": {
                "Subnets": {
                    "10.0.2.0/24": {
                        "IPsInUse": 3,
                        "DynamicIPsAvailable": 253
                    }
                }
            }
        }
    }
]

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Individual_Task
$ docker network rm my_overlay_2
my_overlay_2
```


```
Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Individual_Task
$ docker network rm my_overlay_2
my_overlay_2

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Individual_Task
```

- Попробовать создать сеть host, сохранить результат в отчет.

```
Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Individual_Task
$ docker network create -d host my_custom_host
Error response from daemon: only one instance of "host" network is allowed

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/Individual_Task
```

У компьютера есть только один физический сетевой стек. Docker уже зарезервировал дефолтную сеть `host` для доступа к нему, поэтому создать "виртуальную" копию физической сети невозможно.

Для защиты ЛР необходимо:

- Оформить Отчет со скринами кода в командной строке и результатами его выполнения
- Продемонстрировать работу с командами docker:
  build, **tag**, **run**, **start**, **stop**, **pause**, **unpause**, **restart**, **ps**, **logs**, **ps**, **images**, **network**, **volumes**, **inspect**, а также знать назначение инструкций в **Dockerfile** и **Docker Compose**

ЗАЛИТЬ ОТЧЕТ НА ГИТХАБ!

```
Sanechka@DESKTOP-81JL965 MINGW64 /
$ cd ~/Desktop/453502_KRUCHONOK_13/

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/453502_KRUCHONOK_13 (IGI_LR1)
$ git checkout IGI_LR2
Switched to branch 'IGI_LR2'
Your branch is up to date with 'aleksa/IGI_LR2'.

Sanechka@DESKTOP-81JL965 MINGW64 ~/Desktop/453502_KRUCHONOK_13 (IGI_LR2)
```

Контрольные вопросы

1. Что представляет собой контейнеризация?
2. Что такое Docker?
3. Что является реестром Docker?
4. Что такое Docker Engine?
5. Как происходит связь между клиентом Docker и демоном Docker?
6. Для чего необходим образ контейнера?
7. Что представляет собой контейнер Docker?
8. Каков метод создания Docker-контейнера?
9. Каковы этапы жизненного цикла контейнера Docker?
10. В каких средах можно использовать контейнерные приложения?
    Что такое виртуальная машина?
11. Является ли Docker VM-технологией?
12. Какие различия между виртуальными машинами и контейнерами Docker?
13. Зачем использовать контейнеры?
14. Что выдаст команда $ docker ps ?
15. Сколько идентификаторов по умолчанию имеет каждый контейнер?
16. Как задается имя контейнера?
17. Для чего нужна команда container start?
18. Как создать контейнер без запуска?
19. Что такое запуск контейнеров в интерактивном режиме?
20. Что представляет собой образ?
21. Что такое Dockerfile?
22. Для чего необходим Docker Compose?
23. Какая структура файла Compose?
24. Как запустить несколько контейнеров, используя один сервис?
25. Что такое предоставление порта связанным службам?
26. Для чего используются тома?
27. Для чего применяют переменные среды?
28. Как определяют правила связи между контейнерами?
    Написать файл Docker для создания и копирования каталога и построить его с использованием модулей Python?
29. Какие сети доступны по умолчанию в Docker?
30. Приведите необходимые шаги для развертывания докеризированного приложения, сохраненного в репозитории Git